Technische Universität Dresden

Faculty of Computer Science Biotechnology Center (BIOTEC)

Chair of Bioinformatics

Bachelor Thesis

Embedded Edit Distance: Levenshtein Approximation with a Siamese Neural Network for String Similarity Search

Melissa Analytis

7th September 2026

First referee

Prof. Dr. Michael Schroeder

Second referee

Prof. Dr. Michael Färber

Supervisor

Ferras Mahmoud El-Hendi (M. Sc.)

Technische Universität Dresden

Faculty of Computer Science Chair of Bioinformatics

Abstract

An algorithm's procedure is specified independently of the data on which it is applied. For every valid input, its fixed rules produce an exact, reproducible output with analysable worst-case complexity. A neural networks, by contrast, learns its internal parameters from data, enabling adaption and approximation but providing weaker guarantees and accuracy outside its training distribution. Understanding this tension between both as complementary strengths, motivates asking whether an algorithm's precisely specified target can be combined with a neural network's capacity to learn reusable and potentially transferable representations. Global normalised Levenshtein similarity, derived from the minimum number of edits required to transform one sequence into another, provides a controlled setting for investigating this tension. General-purpose protein embeddings such as ESM already contain a measurable local sequence-similarity signal despite never having been trained for this specific task. This work therefor investigates whether task-agnostic protein embeddings preserve a global similarity signal, whether task-specific supervision strengthens it, and whether the approximation transfers beyond its training distribution.

SNNEED, a 141,184-parameter Siamese encoder, was trained on synthetic string pairs over the 20-symbol amino acid alphabet. It maps each string to a 128-dimensional unit vector and converts the Euclidean distance between two embeddings into a predicted similarity. The frozen encoder was evaluated on independent synthetic strings and on the amino acid, 3Di, and secondary-structure representations of CATH domains. Length ratio and Dice similarity over 3-grams served as non-learned baselines, while frozen ESM-2 embeddings provided a task-agnostic neural reference. Performance was assessed using Spearman rank correlation, AUROC, and MAP@10. The protein-derived representations served only as symbolic evaluation data, no biological property was predicted. ESM-2 contained measurable global similarity information, but the direct supervision of SNNEED strengthened this signal. On 3Di and secondary structure, SNNEED achieved overall Spearman correlations of at least 0.95 and AUROC values above 0.98. MAP@10 nevertheless declined from 0.977 on synthetic strings to 0.502 and 0.405, respectively, indicating meaningful but incomplete transfer.

SNNEED therefore learned a useful, partly transferable approximation of the target relationship, but these results do not not demonstrate that it learned the procedure of the Levenshtein algorithm itself. It may instead approximate its output using transferable statistical properties of the input sequences.

Contents

Abstract III

Symbols and Acronyms XI

1 Introduction 1
- 1.1 Algorithms and Neural Networks 1
- 1.2 Sequence Comparison and Edit Distance 2
- 1.3 Sequence Embeddings 3
- 1.4 Similarity Signals in Protein Language Models 4
- 1.5 Foundations and Research Questions 5
- 1.6 Structure of the Work 7

2 Methods 9
- 2.1 Normalised Levenshtein Similarity 10
- 2.2 SNNEED 11
  - 2.2.1 Encoder 11
  - 2.2.2 Embedding Geometry and Readout 13
- 2.3 Training 14
  - 2.3.1 Synthetic Training Data 14
  - 2.3.2 Training Objective 15
  - 2.3.3 Training Procedure 15
- 2.4 Evaluation Data 16
  - 2.4.1 CATH-S20 16
  - 2.4.2 Synthetically Generated Evaluation Data 16
  - 2.4.3 Sequence Representations and Their Statistics 17
- 2.5 Reference Methods 18
  - 2.5.1 ESM-2 19
  - 2.5.2 Dice similarity Coefficient over 3-Grams 19
  - 2.5.3 Length Ratio 20
- 2.6 Evaluation Protocol 20
  - 2.6.1 Exact Relevance Sets 20
  - 2.6.2 Decile-Balanced Pairwise Evaluation Sets 21
  - 2.6.3 Evaluation-Set Sizes 22

2.6.4 Spearman Rank Correlation . . . 22
2.6.5 AUROC for High-Similarity Discrimination . . . 23
2.6.6 MAP@10 for Retrieval . . . 23
2.6.7 RMSE for Numerical Accuracy . . . 24
2.7 Computational Environment . . . 25
2.7.1 Software . . . 25
2.7.2 Hardware . . . 25

3 Results . . . 27
3.1 Spearman Rank Correlation . . . 27
3.2 Mean Average Precision at Rank Ten . . . 31
3.3 AUROC . . . 33
3.4 Root Mean Squared Error . . . 35
3.5 Conclusion . . . 36

4 Discussion . . . 37
4.1 The Value of Task-Specific Supervision . . . 37
4.2 Transfer Across Representations . . . 39
4.2.1 The Amino-Acid Case . . . 40
4.3 Chance Floor . . . 41
4.4 Architecture and Training Decisions . . . 43
4.4.1 Effect of the Pooling Width . . . 43
4.4.2 Effect of Removing Pooling . . . 44
4.4.3 Effect of the Epoch Budget . . . 45
4.4.4 Effect of the Training-Set Size . . . 46
4.5 Limitations . . . 47
4.6 Future Work . . . 49
4.6.1 A Directly Comparable Baseline Comparison . . . 49
4.6.2 Improving Ranking and Retrieval Quality . . . 49
4.6.3 Transfer Outside Bioinformatics . . . 50

5 Conclusion . . . 51

Acknowledgements . . . 53

A Data Distributions . . . 59

B Composition of the Evaluation Sets . . . 61
B.1 Per-Interval Supply and the Bound . . . 61
B.2 The Number of Candidate Pairs . . . 63
B.3 Size of the Synthetic Evaluation Dataset . . . 63

C The Length Constraint . . . 65

D The Null Datasets . . . 67

List of Figures

1.1 Cosine similarity of ESM-1b embeddings against BLASTp sequence identity 4

2.1 Overview of the approach. The encoder is trained on synthetic string pairs, frozen, and then applied unchanged to every evaluation dataset. 9
2.2 Token look-up 11
2.3 Encoder architecture 12
2.4 Training-pair generation 14

3.1 Overall Spearman rank correlation 28
3.2 Spearman rank correlation by similarity range 28
3.3 ESM-2 cosine similarity against the target 30
3.4 SNNEED cosine similarity against the target 31
3.5 Mean average precision at rank ten 31
3.6 AUROC for high-similarity discrimination 33

4.1 Density of the Tracy–Widom distribution 42
4.2 Spearman correlation across epoch budgets 45
4.3 AUROC across epoch budgets 46
4.4 MAP@10 across epoch budgets 46

A.1 Score distributions of the training and evaluation datasets 59
A.2 Letter-frequency profiles of the evaluation datasets 60
A.3 First-order transition probabilities 60

B.1 Retained pairs per similarity interval 62

D.1 Score distributions against the chance floor, 20-symbol alphabet 68
D.2 Score distributions against the chance floor, 3-symbol alphabet 68

List of Tables

2.1 Similarity ranges on the normalised Levenshtein scale. 11
2.2 Stages of the SNNEED encoder. B denotes the batch size. 12
2.3 Configuration used for every SNNEED training run. 16
2.4 Symbol statistics of the four evaluation datasets. The effective alphabet size is $2^{H_{\text{char}}}$, the number of equally frequent symbols that would carry the same entropy. 17
2.5 Order statistics of the evaluation datasets 18
2.6 High-similarity pairs in each search collection. The counts are exact and cover every pair of sequences within a collection. 21
2.7 Sizes of the three evaluation sets defined in Section 2.6. 22
2.8 Software used 25

3.1 High-range root mean squared error 35

4.1 Pooling width and pooling removal 44
4.2 Training-set size at a fixed epoch budget 47

B.1 Per-interval supply of candidate pairs 62
B.2 Effect of the per-interval bound 62
B.3 Effect of the number of candidate pairs 63
B.4 Effect of the number of independently generated pairs 64

C.1 Sequences removed by each length bound 65

Symbols and Acronyms

3Di	Structural alphabet describing local residue environments
AA	Amino Acid
AUROC	Area Under the Receiver Operating Characteristic Curve
BERT	Bidirectional Encoder Representations from Transformers
BLAST	Basic Local Alignment Search Tool
BLASTp	Protein BLAST
CAFA	Critical Assessment of Functional Annotation
CATH	Class, Architecture, Topology, Homologous Superfamily
CLIP	Contrastive Language-Image Pre-training
CNN	Convolutional Neural Network
CNN-ED	Convolutional Neural Network for Edit Distance
CUDA	Compute Unified Device Architecture
DNA	Deoxyribonucleic Acid
DP	Dynamic Programming
ESM	Evolutionary Scale Modeling
GPU	Graphics Processing Unit
LCS	Longest Common Subsequence
MAP	Mean Average Precision
MMseqs2	Many-against-Many Sequence Searching
MSE	Mean Squared Error
ReLU	Rectified Linear Unit
RMSE	Root Mean Squared Error
S20	Redundancy filter at 20 percent maximum sequence identity
SNNEED	Siamese Neural Network for Embedded Edit Distance
SS	Secondary Structure

1 Introduction

1.1 Algorithms and Neural Networks

At first glance, algorithms and neural networks seem to sit at opposing ends of a spectrum. An algorithm describes a fixed framework, a set of rules and a certain environment, which predicts an almost deterministic outcome. Its objective and procedure are explicitly specified independently of the data on which it is applied. This produces exact and reproducible outputs within its stated domain, and its computational complexity can be analysed. Its behaviour, however, remains tied to its predefined objective, rules and preconditions. Applying it to a different task generally requires these elements to be reworked. A neural network also operates within a predefined architecture and objective, but its internal parameters are learned from data. This allows the network to capture and potentially reuse statistical regularities that were not individually formulated as rules. However, this adaptability comes with weaker guarantees [1]: a network's behaviour is established empirically, and relationships learned from training data may no longer remain reliable outside its training distribution [2].

Understanding this contrast, it becomes clear how algorithms excel at concrete tasks such as sorting lists, traversing graphs, performing arithmetic and comparing sequences because a set of instructions determines and guarantees how their input will be processed. In other words, "algorithms allow us to automate and engineer systems that reason" [1]. For a correctly implemented algorithm, its results are reproducible and its computational complexity can be analysed and assessed in advance, given the inputs satisfy the algorithm's rules. This task-specific nature of an algorithm also presents a real limitation: its objective and procedure do not adapt to a new task without reworking its internal set of rules, and its guarantees extend only as far as the assumptions it was designed around.

Neural networks, by contrast, learn to fit an approximate input–output map and compress objects into a representation that can be reused. They are therefore not bound to the input format an algorithm's preconditions prescribe: where an algorithm requires the richness of a problem, e.g. noisy real-world measurements and circumstances, to be abstracted, a network can operate on a high-dimensional representation directly [1]. In other words, neural networks can absorb noisy conditions that would usually fall outside a pre-defined

procedure's domain, and that makes them potentially robust against task variations. In this, we can find the prospect of "applying classical algorithms on inputs that substantially generalize the preconditions specified by their designers" [1].

Understanding these concrete differences and limitations between algorithms and neural networks helps us motivate why we potentially would want to push a neural network to accomplish what an algorithm can already do. Can the generalisation potential of a learned representation be combined with the specificity of an algorithmically defined target? This ambition is studied more broadly under neural algorithmic reasoning, where neural networks are taught to imitate the workings of an existing algorithm by either reproducing an algorithm's final input–output relationship or in the strongest case by replicating the same intermediate steps [1], with the aim of acquiring more systematic behaviour. The present work approaches this ambition through the first of these two lenses. It does not attempt to extract a runnable algorithm from a neural network or reproduce the internal execution steps of an existing procedure. Instead, it investigates whether a neural network can learn a useful approximation of an algorithm's input–output relationship and how far that approximation survives outside its training distribution. That this reach cannot simply be assumed is the lesson of work on transferring algorithmic reasoning between tasks. There, the expectation that shared algorithmic knowledge lets a network systematically generalise "even when learning only on input-output pairs" is not automatically confirmed [3].

1.2 Sequence Comparison and Edit Distance

Sequence comparison provides a controlled setting for us in which to investigate this tension. Sequences occur throughout computer science and bioinformatics, where they may represent simple text, DNA, proteins or derived structural descriptions. One of the most established measures of sequence dissimilarity is the Levenshtein edit distance [4]. It is defined as the minimum number of insertions, deletions and substitutions required to transform one sequence into another [5]. A dynamic-programming algorithm computes this value exactly for arbitrary symbol strings, with a worst-case time complexity of $\mathcal{O}(nm)$ for sequences of lengths n and m [6]. A smaller edit distance indicates that fewer changes separate the strings and therefore corresponds to greater global similarity. The measure is of interest in biology because its operations are a reasonable proxy for the local mutations by which evolution transforms one sequence into another, and a great deal of biological inference rests on comparing large numbers of sequences [4].

Levenshtein distance is particularly suitable for the present investigation because its value does not depend on the origin or meaning of the symbols. The same operation can be applied to any kind of sequence string, irrespective of its domain: from natural-language text strings to biological sequences such as natural amino-acid strings, and to symbolic descriptions of protein shape, in which every position of a structure is replaced by a symbol. Such descriptions include secondary-structure strings, which label each position as a helix, a strand or a loop, and 3Di strings, whose twenty-symbol structural alphabet describes for each position the geometric conformation it forms with its spatially closest neighbour [7]. Normalising the distance by the length of the longer sequence additionally produces a common similarity scale for strings of different sizes. The resulting global, normalised

Levenshtein similarity therefore provides a consistent and exactly computable target across differing sequence distributions. This makes it possible to ask whether a neural network preserves transferable properties of the target function or merely captures regularities of the sequence strings on which it was trained.

This global edit distance target must be distinguished from other measures of sequence resemblance, in two respects. The first concerns scope. In bioinformatics, tools such as BLASTp [8] search for strong local alignments: two proteins may share a similar region or domain even when their complete sequences differ substantially. Global normalised Levenshtein similarity instead evaluates the edits required across the complete strings. The second concerns the operations that are counted, because edit distance names a family of procedures rather than a single one. The Longest Common Subsequence (LCS) edit distance measures the minimum number of insertions and deletions needed to turn one sequence into another [9]. Like Levenshtein, it is computed over the complete strings, but it forbids substitution. This thesis therefore fixes one member of that family: the global, unit-cost Levenshtein distance. These measures are related through their common concern with sequence resemblance, but they all answer different questions. Existing tools such as MMseqs2 [10] already support large-scale biological sequence search. The purpose of this thesis is therefore not to present neural embeddings as a solution to an otherwise unsolved search problem. Rather, global Levenshtein similarity is used as a controlled algorithmic ground truth against which the geometry and transfer behaviour of learned representations can be measured.

1.3 Sequence Embeddings

The learned representations in question are embeddings. A learned map $f\colon \mathcal{X} \to \mathbb{R}^d$ assigns to each object $x \in \mathcal{X}$ a vector $f(x)$ of fixed dimension d. The term *embedding* is used in the literature both for this map and for the vector it produces. This thesis reserves it for the vector $f(x)$, and calls the map itself the *encoder*. If the encoder preserves a chosen relationship, objects that are similar under that relationship are positioned close to one another in $\mathbb{R}^d$. Embeddings have become important representations for text, images and biological sequences. BERT [11] learns contextual representations of language, CLIP [12] relates images and text in a shared representation space, and protein language models such as ProtTrans [13] and ESM [14] learn representations from large collections of amino-acid sequences. The protein models are trained with self-supervised objectives such as masked language modelling, which makes them task-agnostic: their parameters are shaped by the regularities of the sequence data rather than by any particular downstream target. They are not trained specifically to reproduce global normalised Levenshtein similarity. Because the regularities that shape them are properties of the sequences themselves, however, their geometry may still reflect how similar two sequences are, without any similarity target having been optimised.

1.4 Similarity Signals in Protein Language Models

Evidence that such a signal already exists was provided by a comparison between protein-language-model embeddings and BLASTp sequence identity [15]. Identity is a useful point of reference here because, like normalised Levenshtein similarity, it is a normalised count of matching positions rather than an alignment score. The two differ in scope rather than in what they count: the identity reported by BLASTp is computed over a local alignment, whereas normalised Levenshtein similarity is computed over the complete strings. Two further measures recur throughout that comparison and in the present work. Cosine similarity compares two vectors $u, v \in \mathbb{R}^d$ by the angle between them, $\cos(u, v) = \langle u, v \rangle / (\|u\| \, \|v\|)$, so that 1 indicates identical direction and 0 orthogonality. Spearman rank correlation measures how closely the ordering of one set of values matches the ordering of another. It takes the value 1 when two orderings match perfectly and -1 when they are exactly opposite to each other.

On a subset of CAFA3 [16], a protein function prediction benchmark comprising human proteins, cosine similarities derived from ESM-1b [17] representations reached a Spearman rank correlation of approximately $\rho = 0.66$ with BLASTp sequence identity, the largest coefficient among the 12 representations compared [15]. ProtTrans placed second, at 0.63, with the remaining ten falling away to 0.27. The model evaluated there is ESM-1b; the one used in this work is its successor, ESM-2 [14]. The correlation is monotone and a similarity signal therefore present, but the underlying values are severely compressed: cosine similarity occupies only the upper end of its range across the whole identity axis, with most pairs above 60% sequence identity falling between 0.95 and 1.00 cosine similarity, irrespective of their BLASTp identity (Figure 1.1).

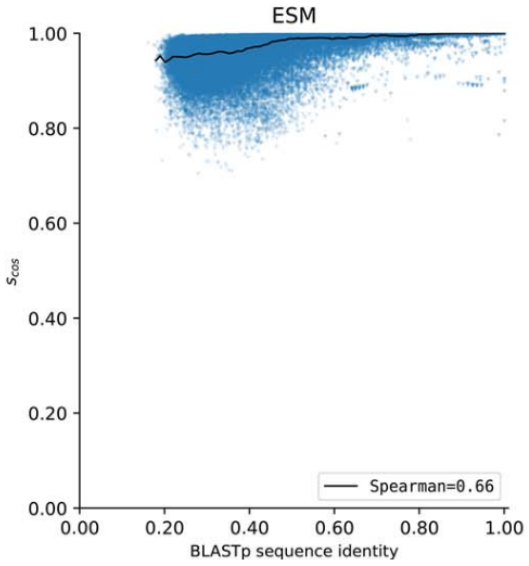


Figure 1.1: Cosine similarity between ESM-1b embeddings plotted against BLASTp sequence identity on the CAFA3 subset. The black curve is the interval average from which the Spearman coefficient of 0.66 was computed. Cosine values are confined to the upper end of their range across the entire identity axis, which is the compression described in the text. Image from [15, p. 25, Fig. 6].

A general, task-agnostic protein language model is evidently not blind to sequence similarity, even though it was never trained for that comparison. The ordering it induces is nevertheless

coarse, and it was measured against a different target: BLASTp evaluates local alignment relationships, over a different protein pool. The value $\rho = 0.66$ is therefore not a performance level that ESM-2 should be expected to reproduce here. However, what it establishes is more limited but still important: the geometry of a general-purpose protein embedding space contains a measurable sequence-similarity signal, and the question taken up here is whether direct, task-specific supervision on a global string-similarity target can sharpen it.

1.5 Foundations and Research Questions

This observation creates the opening addressed by the present work. If local similarity is already reflected in a task-agnostic representation, the same representation may also carry a global one, and a small network trained directly on global, normalised Levenshtein similarity may sharpen it further. Whether it still does so once it is moved to a different data distribution is a separate matter, and it is the one that distinguishes performance on familiar data from abstraction beyond the training distribution.

Several design choices adopted here are inherited rather than invented. CNN-ED [18] proposes an approach that maps strings into a 128-dimensional Euclidean space and trains vector distances to approximate edit distance. Its encoder pipeline of convolution, pooling and linear layer is the one adopted here: one-dimensional convolutions "allow the same character in different positions to interact with each other, which corresponds to insertion and deletion in edit distance computation" [18, Sec. 3.1], while pooling preserves a bound on edit distance and converts feature maps of varying sizes into fixed-length summarisations [19, Sec. III.A], the role played here by adaptive average pooling. NeuroSEED [20] develops the same principle for biological sequences and investigates how different encoders and embedding geometries preserve sequence distances. Their central idea is that the distance between two embeddings is itself the prediction, so that "the distance function d between the embedded points defines the geometry of the embedding space" [20, Sec. 3]. It compares Euclidean with alternative geometries, and that comparison is what motivates the metric space adopted here.

Inspired by their work, this work proposes SNNEED, a small task-specific Siamese encoder for global normalised Levenshtein similarity. SNNEED processes both members of a sequence pair with a single shared-parameter encoder, in the Siamese arrangement [21], and converts the Euclidean distance between two 128-dimensional unit vectors directly into a predicted similarity, with no learned prediction head on top of the pair. The conversion from a distance to a similarity in $[0, 1]$, specified in Section 2.2.2, is itself a choice of geometry reflecting NeuroSEED's finding that the supervised relationship must be expressed in the geometry that is later used for evaluation and retrieval. NeuroSEED's k-mer baseline, which measures how many length-k substrings two sequences share, is the fixed counterpart to a learned encoder [20, Sec. 2] and motivates the Dice coefficient over 3-grams used here as a baseline. Chapter 2 specifies these components without repeating their justification.

SNNEED is trained on synthetically generated sequence pairs using exact global normalised Levenshtein similarity as supervision. After training, its parameters are frozen and the same encoder is evaluated on independent, synthetically generated strings, natural amino-acid sequences, and protein-derived 3Di and secondary-structure representations. These

distinct data sets differ in their similarity-score distributions, character frequencies and order-dependent sequence patterns. The protein-derived representations are used as symbolic evaluation distributions. No claim is made that unit-cost Levenshtein similarity predicts protein homology, structure or function.

Both predecessors, CNN-ED and NeuroSEED, fit one model per dataset and task, so their reported evaluations do not establish how one frozen encoder behaves once the data distribution changes, which is the transfer question taken up here. That this transfer behaviour cannot simply be assumed is the lesson of work on transferring algorithmic reasoning between tasks, where neither fine-tuning nor freezing parameters produced systematic transfer, and only multi-task training across related algorithms did [3].

The frozen representation of ESM-2 provides a task-agnostic point of comparison. On amino-acid sequences, it tests whether global Levenshtein sequence similarity information is already present in a general protein representation. On 3Di and secondary-structure strings, both ESM-2 and SNNEED receive symbols whose statistics differ from those they were trained on. Synthetic strings are a different case: they lie outside the distribution ESM-2 was pre-trained on, but they are the distribution SNNEED was trained on. The outputs of either model in these settings are therefore not statements about biological understanding. Instead, they serve as controls for whether any useful string-similarity geometry survives when the character statistics and representation change. Two non-learned comparisons complete the set, both specified in Chapter 2: the Dice coefficient over 3-grams, which establishes how far sequence pairs can be ordered from shared substrings of length three alone, and the length ratio, which establishes how much of that ordering follows from sequence length by itself.

Finally, the aim is to approximate the input–output mapping of the Levenshtein algorithm rather than to supervise a network with the algorithm's own intermediate steps [1, p. 2]. The latter has been pursued to its strongest form by Ohtomo et al. [22], who construct a network that computes the Levenshtein distance exactly. Their reported learning experiments, which recover a target string from a set of strings and their distances to it, stall for that distance. Exact neural realisation is therefore possible by construction, while the learning evidence remains inconclusive. This sharpens the scope of this thesis, making approximation a deliberate property of the problem formulation.

The thesis is guided by two research questions:

Q1 — Signal and task-specific improvement. A general, task-agnostic protein encoder already carries a local similarity signal. Does ESM-2 also carry a global similarity signal, and can a task-specific encoder trained directly on that target yield a stronger signal?

Q2 — Abstraction and transfer. Can SNNEED abstract beyond its training distribution? Trained on one data distribution and tested on another, did it learn transferable properties of the algorithm's output relationship, and how far do those properties survive, or did it learn merely regularities of the training data?

The second question is intentionally posed as a tension rather than a simple yes/no-answer. Transfer without retraining would provide evidence that the approximation is not restricted to the exact training distribution, but it would not show that SNNEED recovered the dynamic-programming procedure, only that it learned an input–output mapping consistent with that procedure. The network may instead rely on transferable cues such as length, composition or

local patterns to approximate the algorithm's output. Consequently, this work operationalises abstraction as the degree and regime in which a similarity signal survives distribution shift, rather than as a claim that the algorithm itself has been learned.

To answer these questions, task-agnostic and task-specific representations are compared through complementary views of the predicted similarity geometry. Rank correlation measures whether the overall ordering of sequence similarities is retained, high-similarity discrimination tests whether close pairs can be separated from the remaining pool, and retrieval evaluation tests whether highly similar partners can be recovered for a query. The trained model is then frozen while the input distribution moves from synthetic training strings to natural amino-acid sequences and to 3Di and secondary-structure representations. Chapter 2 defines the representations, targets and evaluation measures, and specifies the model and experimental protocol.

1.6 Structure of the Work

The remainder of this thesis is organised as follows. Chapter 2, Methods, specifies the target function, the synthetic training data, the evaluation data, the encoder architecture, the training objective and the evaluation protocol. Chapter 3, Results, reports approximation quality across the evaluation feeds and decomposes it by similarity band. Chapter 4, Discussion, establishes the chance floor against which those results are read, interprets them, examines the architectural and training choices behind them through a series of controlled variations, and states the limitations of the study and the directions it leaves open. Chapter 5, Conclusion, summarises what the work establishes. Three appendices follow, covering the data distributions, the composition of the evaluation sets, and the length constraint.

2 Methods

Section 1.5 has elaborated why the components described here were chosen. This chapter states what was built: the encoder, the data it was trained and evaluated on, the reference methods it is compared against, and the evaluation protocol.

The approach consists of three phases, shown in Figure 2.1.

In the first phase, the encoder is trained exclusively on synthetically generated string pairs. Both members of a pair are processed by the same network, and the predicted similarity is derived from the distance between the two resulting vectors and compared against the exact normalised Levenshtein similarity of that pair. No biological sequence is used at any point during training.

In the second phase, the trained parameters are frozen and every sequence of every evaluation dataset is encoded once into a fixed-size vector. No adaptation of any kind is performed on an evaluation dataset.

In the third phase, the resulting vectors are evaluated in two complementary ways: as a ranking of sequence pairs, and as a retrieval of nearest neighbours from the complete collection of encoded sequences. The pairs and queries are drawn once per evaluation dataset and used for every method.

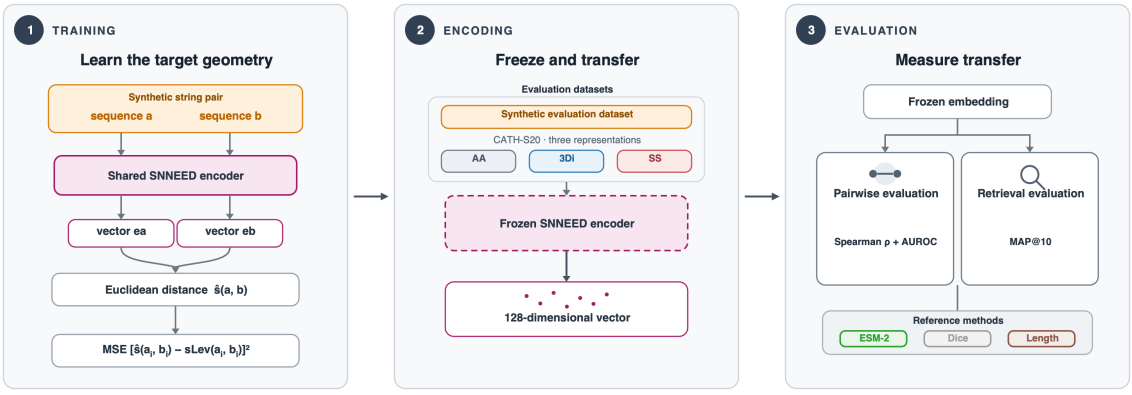


Figure 2.1: Overview of the approach. The encoder is trained on synthetic string pairs, frozen, and then applied unchanged to every evaluation dataset.

The remainder of this chapter follows that order. Section 2.1 defines the target quantity and the ranges used throughout the evaluation. Section 2.2 specifies the encoder and the geometry of the space it produces. Section 2.3 describes the synthetic training data, the objective and the training procedure. Section 2.4 specifies the evaluation datasets, and Section 2.5 the reference methods. Section 2.6 defines the evaluation sets and the metrics computed on them, and Section 2.7 states the software and hardware used.

2.1 Normalised Levenshtein Similarity

All supervision and all ground truth in this thesis derive from a single quantity. For two strings a and b over a common alphabet Σ, let $d_{\mathrm{Lev}}(a, b)$ denote the global Levenshtein distance under unit costs, in which each insertion, deletion and substitution contributes a cost of one [4]. The normalised Levenshtein similarity of the pair is

$$s_{\mathrm{Lev}}(a, b) \;=\; 1 \;-\; \frac{d_{\mathrm{Lev}}(a, b)}{\max\big(|a|, |b|\big)}, \tag{2.1}$$

where $|a|$ and $|b|$ denote the lengths of the two strings. Because a string of length $|a|$ can always be transformed into a string of length $|b|$ using at most $\max(|a|, |b|)$ operations, the distance is bounded by the length of the longer string and therefore $s_{\mathrm{Lev}}(a, b) \in [0, 1]$. A value of 1 denotes identical strings, and a value of 0 denotes the maximum dissimilarity this metric can express, namely the number of required edits equals the length of the longer string.

The denominator in Equation 2.1 is the length of the longer of the two strings. This keeps the unit-cost score inside $[0, 1]$; normalising by the shorter string can yield values below zero.

The quantity is length-dependent by construction. Reconciling two strings of unequal length requires at least $\big||a| - |b|\big|$ insertions or deletions, so $d_{\mathrm{Lev}}(a, b) \geq \big||a| - |b|\big|$ and consequently

$$s_{\mathrm{Lev}}(a, b) \;\leq\; \frac{\min\big(|a|, |b|\big)}{\max\big(|a|, |b|\big)}. \tag{2.2}$$

Two strings of very different lengths cannot reach a high similarity, whatever their contents. This is a property of the target rather than of any method that approximates it.

Reference values are computed with RapidFuzz (version 3.14.5) [23], which implements the exact global unit-cost Levenshtein distance. No approximate variant is used at any point, so every value of s_{Lev} reported in this thesis is exact.

Table 2.1 defines three ranges on the s_{Lev} scale that are used throughout the evaluation.

The high range defines the set of relevant neighbours for retrieval (Section 2.6). Both boundaries are applied identically to every evaluation dataset and every method. They are not homology criteria and carry no biological claim.

Table 2.1: Similarity ranges on the normalised Levenshtein scale.

Range	Definition
Far	$s_{\text{Lev}} < 0.30$
Mid	$0.30 \leq s_{\text{Lev}} < 0.70$
High	$s_{\text{Lev}} \geq 0.70$

2.2 SNNEED

2.2.1 Encoder

SNNEED uses a Siamese architecture in which a single encoder f_θ is applied to both sequences of a pair [21]. For a pair (a, b), the two resulting representations are

$$e_a = f_\theta(a), \qquad e_b = f_\theta(b). \tag{2.3}$$

The two branches therefore do not mean separate networks: both use exactly the same encoder and parameters. The encoder is the complete learned mapping from an input sequence to its final 128-dimensional representation. SNNEED contains no additional learned prediction head after the two encodings.

Each input sequence is first mapped character-wise to a sequence of integer indices. Each index then selects one row of a learned look-up table with 21 rows: 20 rows for the alphabet symbols and one row reserved for padding. A non-padding index selects a 32-dimensional row that is updated during training. The padding row marks positions that do not belong to the original sequence and is excluded from parameter updates. Figure 2.2 illustrates this mapping.

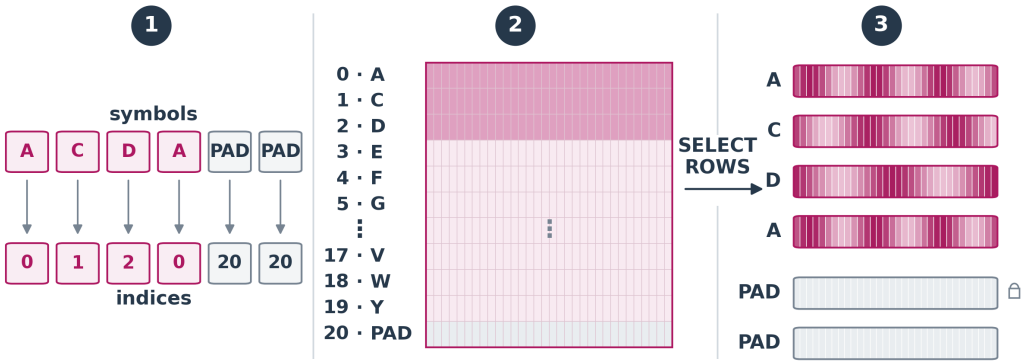


Figure 2.2: Token look-up, shown for the example sequence `ACDA` padded here to six positions. **(1)** Symbols to indices: each symbol is replaced by an integer index and the model receives this integer sequence directly; the repeated `A` selects row 0 a second time. **(2)** Learned look-up table: 21 rows and 32 columns, with 32 feature values in every row. Rows 0–19 are updated during training, row 20 is reserved for padding and receives no gradient. **(3)** Selected dense rows: the look-up returns one 32-dimensional vector per input position, giving 6×32 values for this example, in which the two `A` positions return identical vectors.

All sequences are right-padded to a common width of 200 positions. This width corresponds to the upper bound applied when constructing the training and evaluation datasets (Section 2.4). Sequences longer than 200 symbols were excluded before encoding and

were not truncated. The secondary-structure alphabet $\{\mathrm{H}, \mathrm{L}, \mathrm{S}\}$ and the 3Di alphabet use characters contained in the same 20-symbol index set. Amino-acid, secondary-structure and 3Di sequences can therefore be processed by the same look-up table without a representation-specific token or encoder.

The selected 32-dimensional token vectors are passed through two one-dimensional convolutional layers. The first convolution produces 32 channels and the second produces 64 channels. Both use a kernel width of three, pad one position at each boundary, and are activated with a ReLU function. Activations at padded positions are then set to zero.

Adaptive average pooling then reduces the 200 positions of each channel to 16 averaged windows. The number of output positions is fixed at 16 regardless of the input width. Because every sequence is padded to the same width of 200, the windows are identical for every input. The resulting 64×16 feature map is flattened into 1,024 values and projected linearly to 128 dimensions. Finally, the projected vector is L2-normalised. Figure 2.3 summarises this sequence of operations while Table 2.2 gives the corresponding dimensions and parameter counts.

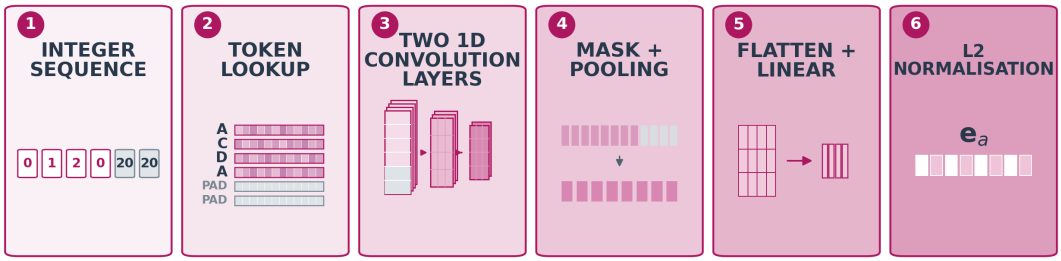


Figure 2.3: Architecture of the shared SNNEED encoder, in the six stages of Section 2.2.1: **(1)** integer sequence, right-padded to 200; **(2)** token look-up, 21 rows of 32 values; **(3)** two one-dimensional convolutions, $32 \to 32 \to 64$ channels, kernel width three, ReLU; **(4)** padded positions zeroed, then averaged into 16 windows; **(5)** the 64×16 map flattened to 1,024 values and projected to 128; **(6)** L2 normalisation to the unit vector e_a. The same encoder and the same parameters are used for both sequences of every input pair.

Table 2.2: Stages of the SNNEED encoder. B denotes the batch size.

Stage	Specification	Output shape	Parameters
Input	Integer indices, right-padded to width 200	$B \times 200$	0
Token lookup	21 rows, 32 dimensions; axes rearranged for convolution	$B \times 32 \times 200$	672
Convolution 1	32 to 32 channels, kernel width 3, ReLU	$B \times 32 \times 200$	3,104
Convolution 2	32 to 64 channels, kernel width 3, ReLU	$B \times 64 \times 200$	6,208
Padding mask	Padded positions set to zero	$B \times 64 \times 200$	0
Adaptive pooling	Average pooling to 16 positional windows	$B \times 64 \times 16$	0
Flattening	$64 \times 16 = 1{,}024$ features	$B \times 1{,}024$	0
Projection	Linear map from 1,024 to 128 dimensions	$B \times 128$	131,200
Normalisation	L2-normalisation along the feature dimension	$B \times 128$	0
		Total	141,184

2.2.2 Embedding Geometry and Readout

The final step of the encoder is an L2-normalisation: each vector is divided by its own L2 norm, also known as the Euclidean norm, $\|v\|_2 = \sqrt{\sum_{i=1}^{128} v_i^2}$, so that the result has Euclidean length one, $\|e\|_2 = 1$. Consequently, every encoded sequence lies on the unit sphere in $\mathbb{R}^{128}$, the 128-dimensional Euclidean space. Two sequences are compared by the Euclidean distance between their embeddings:

$$d_{\mathrm{Euc}}(a, b) = \|e_a - e_b\|_2. \tag{2.4}$$

Because both points lie on the sphere, this is the length of the chord joining them rather than the arc along the surface. Because both vectors have length one, it takes values in $[0, 2]$. The distance is divided by two so that it spans the same interval as the normalised Levenshtein similarity (Equation 2.1), and subtracted from one because a distance and a similarity run in opposite directions: a distance of zero denotes identical sequences, which the target scores as one. SNNEED converts the Euclidean distance into a predicted similarity using

$$\widehat{s}(a, b) = 1 - \frac{d_{\mathrm{Euc}}(a, b)}{2} = 1 - \frac{\|e_a - e_b\|_2}{2}. \tag{2.5}$$

This maps the Euclidean distance to the interval $[0, 1]$. The readout itself has no trainable parameters; the encoder is therefore the only learned component. The readout is not cosine similarity, but the two are closely related. Because both embeddings have unit length, expanding the square gives

$$\|e_a - e_b\|_2^2 = 2 - 2\cos(e_a, e_b), \tag{2.6}$$

where $\cos(e_a, e_b)$ denotes the cosine similarity of the two embeddings, which for unit vectors is their inner product. The Euclidean distance is therefore a strictly decreasing function of the cosine similarity, and the predicted similarity a strictly increasing one: as the cosine of a pair increases, so does its predicted similarity, and ranking pairs by $\widehat{s}$ and ranking them by cosine similarity give the same order.

The two are used for different purposes. Rank-based measures, where only the ordering of pairs or candidates matters, are computed from cosine similarity. Across methods only these orderings are compared, because the reference methods produce similarity values on their own numerical scales. The Euclidean readout is used where the predicted value itself is compared against s_{Lev}: it is the quantity the training objective regresses (Section 2.3) and the quantity entering the root mean squared error (Section 2.6).

2.3 Training

2.3.1 Synthetic Training Data

SNNEED is trained exclusively on synthetically generated string pairs. Three requirements are defined for the synthetic training dataset:

- every normalised Levenshtein similarity (Equation 2.1) that a pair of sequences can reach must be exact and available across the whole similarity range, not only where natural pairs happen to fall;
- the similarity distribution must be controllable, since the natural distribution of the chosen evaluation datasets is concentrated below 0.30 (Section 2.4);
- no biological sequence may enter training, so that every evaluation is a shift away from the training distribution and constitutes a test for transfer as defined in Q2 (Section 1.5).

For each random seed a new training dataset is generated and a new encoder is trained from random initialisation. A given seed reproduces the same training set.

Generation process

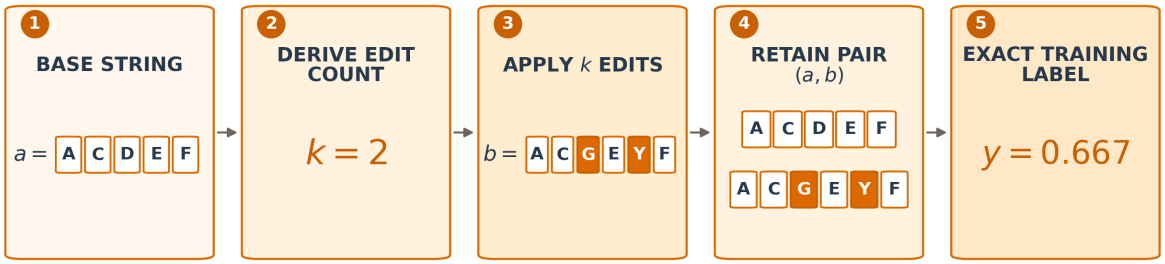


Figure 2.4: Generation of one training pair (Section 2.3.1). The sampled target similarity t sets only the number of edits k; the label y is the exact normalised Levenshtein similarity recomputed from the realised pair, here $1 - 2/6 = 0.667$.

A base string is drawn by sampling its length L uniformly, $L \sim \mathcal{U}\{50, \ldots, 200\}$. Each of its L symbols is then drawn independently and uniformly from the 20-symbol amino-acid alphabet $\Sigma = \{\mathrm{A}, \mathrm{C}, \mathrm{D}, \mathrm{E}, \mathrm{F}, \mathrm{G}, \mathrm{H}, \mathrm{I}, \mathrm{K}, \mathrm{L}, \mathrm{M}, \mathrm{N}, \mathrm{P}, \mathrm{Q}, \mathrm{R}, \mathrm{S}, \mathrm{T}, \mathrm{V}, \mathrm{W}, \mathrm{Y}\}$. A target similarity t is sampled from $\mathcal{U}(0, 1)$ and converted into a number k of edit operations:

$$k = \mathrm{round}\big((1 - t)L\big), \tag{2.7}$$

so that the number of operations is drawn uniformly over the length of the base string.

Each of the k operations is applied sequentially to the current string and is drawn uniformly from the three Levenshtein edit operations: substitution, insertion and deletion. A substitution replaces a symbol by a different symbol drawn uniformly from the remaining 19, an insertion places a uniformly drawn symbol at a uniformly drawn position, and a deletion

removes the symbol at a uniformly drawn position. At the upper length limit of 200, only deletion and substitution are allowed.

After all edits have been applied, the training label is recomputed from the base string and its altered partner as the exact normalised Levenshtein similarity of that realised pair (Section 2.1). The label is therefore not t: edit operations can coincide or cancel, so the realised similarity differs from the value that determined k.

30,000 pairs are generated per seed. The realised labels are not uniformly distributed over $[0, 1]$, which is examined further in Section 4.3.

2.3.2 Training Objective

SNNEED is trained by minimising the mean squared error between the predicted similarity and the exact normalised Levenshtein similarity of the pair. Let

$$\mathcal{D}_{\text{train}} = \{(a_i, b_i, s_{\text{Lev}}(a_i, b_i))\}_{i=1}^{N} \tag{2.8}$$

denote a training dataset containing $N = 30{,}000$ pairs. For a pair (a_i, b_i), SNNEED produces the prediction $\widehat{s}_\theta(a_i, b_i)$ using the parameter-free Euclidean-distance readout defined in Equation 2.5. The encoder parameters θ are optimised by minimising the mean squared error

$$\mathcal{L}_{\text{MSE}}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \left(\widehat{s}_\theta(a_i, b_i) - s_{\text{Lev}}(a_i, b_i)\right)^2 . \tag{2.9}$$

Every training pair contributes equally to the objective. Since the readout itself contains no trainable parameters, optimisation affects only the shared encoder.

2.3.3 Training Procedure

The model is trained with the Adam optimiser [24] using a fixed learning rate of 10^{-3}, with a batch size of 128, for 30 epochs over the 30,000 training pairs. No separate validation split is held out for evaluation, and the parameters after the final epoch are used.

Training is repeated for three seeds (0, 1 and 2), each controlling both the generation of the training pairs and the initialisation of the parameters. SNNEED results are reported as the arithmetic mean and standard deviation over these three independently trained encoders. The ESM-2 and Dice reference methods do not require training in this study and are therefore evaluated once. Table 2.3 summarises the configuration.

Table 2.3: Configuration used for every SNNEED training run.

Setting	Value
Training pairs	30,000 per seed
Objective	Unweighted mean squared error
Optimiser	Adam
Learning rate	10^{-3}
Batch size	128
Epochs	30
Random seeds	0, 1 and 2

2.4 Evaluation Data

2.4.1 CATH-S20

The biological evaluation data derive from CATH-S20, release 4.3.0, downloaded in April 2026 [25]. S20 denotes a set of representative protein domains which are redundancy-reduced to a maximum of 20% sequence identity.

All unique domains are yielding 14,907 domains before filtering. No model is trained on CATH, so the partitioning has no function here. Each representation is processed as a stand-alone dataset, and filtering is applied separately to each. A sequence is retained if its length lies between 50 and 200 symbols and every symbol belongs to the alphabet defined for that representation: the 20 canonical amino-acid letters for AA and 3Di, as stated in Section 2.3.1, or $\{\mathrm{H}, \mathrm{L}, \mathrm{S}\}$ for SS. Restricting the length interval is discussed further in Appendix C. Sequences longer than 200 symbols are excluded rather than truncated, and behaviour on longer inputs is therefore neither trained nor evaluated.

The evaluation reported here contains an explicit exception to this rule. The domains `4z0mC02` and `3qkaE02` were retained outside the length interval after they were identified as contributing high-similarity AA pairs. Including these domains produces search collections of 10,501 AA sequences, 10,497 SS sequences and 10,501 3Di sequences. Without the exception, the corresponding counts are 10,499, 10,495 and 10,499. The exception is stated explicitly because it affects the number of available high-similarity AA pairs. Its consequences for the evaluation are described in Section 2.6.

2.4.2 Synthetically Generated Evaluation Data

This dataset is generated synthetically from the same generator as the training data in Section 2.3.1, so that in-distribution behaviour can be tested. Going forward it is referred to as *Synth*. The synthetic training and evaluation sets are drawn from that generator but are disjoint samples.

Synth combines two kinds of pair. In 20,000 pairs, a base string is generated and altered as described in Section 2.3.1. In the remaining 8,000 pairs, both strings are generated independently of one another. The effect of this number on the composition of the dataset is reported in Appendix B, Table B.3.

The independent pairs populate the low similarity range, which altered pairs reach only rarely: without them the two lowest deciles of the similarity scale are empty. Why that range is otherwise unreachable, and what it implies for how the evaluation sets are composed, is examined in Section 4.3. How evaluation pairs are used from this dataset is described in Section 2.6.

2.4.3 Sequence Representations and Their Statistics

The remaining three datasets are obtained from the same collection of protein domains but express each domain as an amino-acid (AA), secondary-structure (SS) or 3Di sequence.

The AA representation is the same amino-acid sequence of a domain. The SS representation describes the same domain using the three symbols H, L and S. The 3Di strings are from the corresponding structures. Its structural alphabet contains 20 symbols represented by the same letters used for the amino-acid alphabet. Synth shares its alphabet with AA and 3Di.

All four representations can therefore be processed by the same 21-row SNNEED look-up table described in Section 2.2.1. They are nevertheless treated as separate evaluation datasets.

The representations differ in both alphabet size and symbol distribution. This difference is quantified using the character entropy over all symbols in their respective alphabets:

$$H_{\mathrm{char}} = -\sum_{c \in \Sigma} p(c) \log_2 p(c), \tag{2.10}$$

where $p(c)$ is the relative frequency of symbol c in the corresponding alphabet. Because the alphabets differ in size, H_{char} is not comparable across representations on its own. The comparable quantity is the normalised character entropy

$$\tilde{H} = \frac{H_{\mathrm{char}}}{\log_2 |\Sigma|}, \tag{2.11}$$

where $|\Sigma|$ is the number of symbols in the alphabet. A value of 1 means every symbol occurs equally often, and lower values mean that the mass concentrates on fewer symbols. Table 2.4 reports both quantities.

Table 2.4: Symbol statistics of the four evaluation datasets. The effective alphabet size is $2^{H_{\mathrm{char}}}$, the number of equally frequent symbols that would carry the same entropy.

Dataset	$\lvert\Sigma\rvert$	H_{char}	$\tilde{H}$	Effective alphabet	Most frequent
Synth	20	4.322	1.000	20.00	all letters uniform (0.050)
AA	20	4.162	0.963	17.91	L (0.098)
3Di	20	3.800	0.879	13.93	V (0.227)
SS	3	1.524	0.962	2.88	L (0.420)

Measured against its own alphabet, SS is as uniform as AA. The concentrated representation is 3Di, whose 20 symbols carry the entropy of about 14 equally frequent ones. The Synth

values characterise the generator that both synthetic sets share rather than any single collection.

The quantities above describe how often each symbol occurs, not how symbols follow one another. A sequence in which the same symbol repeats in runs carries less information per position than its character entropy alone suggests. The first-order conditional entropy

$$H_{\mathrm{cond}} = -\sum_{a,b\in\Sigma} p(a,b)\,\log_2 p(b \mid a) \tag{2.12}$$

measures the entropy of a symbol given the symbol preceding it, and the difference $I = H_{\mathrm{char}} - H_{\mathrm{cond}}$ is the first-order mutual information: the number of bits per position that the preceding symbol already determines. Table 2.5 reports both, and Figure A.3 shows the underlying transition probabilities.

Table 2.5: First-order order statistics of the four evaluation datasets. H_{cond} is the entropy of a symbol given the preceding one, and I is the mutual information between adjacent positions. Both are measured in bits per position.

Dataset	H_{cond}	I
Synth	4.322	0.000
AA	4.155	0.007
3Di	3.249	0.551
SS	0.649	0.875

The synthetic generator emits symbols independently, so its mutual information is zero by construction, and amino-acid sequences are almost as unconstrained at this order, at 0.007 bits. The two protein-derived representations differ: 0.551 bits for 3Di and 0.875 bits for SS, the latter more than half of that representation's character entropy of 1.524 bits. Both quantities are first order and describe only the dependence between adjacent positions.

The four datasets also differ in the distribution of the target quantity itself. Computed over all pairs of each search collection, the median normalised Levenshtein similarity is 0.198 for AA, 0.238 for 3Di and 0.438 for SS. The pairs of Synth are generated rather than drawn from a collection, and their median is 0.512. The three protein-derived datasets therefore concentrate in the far and mid ranges of Table 2.1, so that pairs in the high range make up only a small fraction of each of them. The full distributions are shown in Appendix A.

The medians above describe the natural pair distributions of each dataset, before any evaluation set is drawn from them. Each dataset concentrates in a different part of the scale. Section 2.6 defines how evaluation pairs are drawn from these distributions.

2.5 Reference Methods

SNNEED is compared with three reference methods. ESM-2 [14] provides a task-agnostic, learned representation (Section 1.5), while the Dice coefficient and the length ratio provide non-learned comparisons.

The resulting similarity scores are not assumed to share the same numerical scale. Spearman correlation, AUROC and MAP@10 therefore compare the ordering induced by each method. Absolute prediction error against s_{Lev} is evaluated only for SNNEED, whose readout is trained directly on that scale.

2.5.1 ESM-2

The ESM-2 reference uses the pretrained 12-layer, 35-million-parameter model `esm2_t12_35M_UR50D` [14]. Its parameters remain frozen and no fine-tuning or other adaptation is performed on any evaluation dataset.

Each sequence is processed using the model's own tokeniser. The padding and the beginning- and end-of-sequence tokens are excluded. ESM-2 produces one context-dependent vector for each sequence position. These vectors are averaged to map the complete sequence s to a single embedding. This embedding is then L2-normalised, using the same operation defined for SNNEED in Section 2.2.2. It therefore lies on the unit sphere, allowing that two sequences a and b are scored using their cosine similarity. Because the vectors are L2-normalised, this is equal to their dot product:

$$s_{\mathrm{ESM}}(a, b) = \cos(\mathbf{e}_a, \mathbf{e}_b) = \mathbf{e}_a^{\top} \mathbf{e}_b. \tag{2.13}$$

Unlike in SNNEED, this normalisation is only applied during evaluation and is not part of task-specific training.

On amino-acid sequences ESM-2 serves as the task-agnostic baseline. On SS and 3Di sequences it serves as a transfer control, their symbols being interpreted through its amino-acid tokeniser.

2.5.2 Dice similarity Coefficient over 3-Grams

The second reference method represents each sequence by its set of distinct, neighbouring 3-grams. For a sequence $s = s_1 s_2 \dots s_n$, this set S is

$$S_3(s) = \{ s_i s_{i+1} s_{i+2} \mid 1 \le i \le n - 2 \}. \tag{2.14}$$

Repeated occurrences of the same 3-gram are included only once. The similarity between two sequences is then calculated using the Dice coefficient [26]:

$$s_{\mathrm{Dice}}(a, b) = \frac{2\,|S_3(a) \cap S_3(b)|}{|S_3(a)| + |S_3(b)|}. \tag{2.15}$$

The score lies in $[0, 1]$. A value of one indicates that both sequences contain the same set of distinct 3-grams, whereas a value of zero indicates that their sets do not overlap and the two sequences are therefore maximally dissimilar. The baseline therefore establishes how well sequence pairs can be ordered by similarity from the number of shared substrings of length

three alone, without any learned representation. The same definition is applied unchanged to all four datasets. For retrieval, candidates are ordered from the highest to the lowest Dice coefficient.

2.5.3 Length Ratio

The length-ratio baseline assigns similarity using only the lengths of the two sequences:

$$s_{\text{length}}(a, b) = \frac{\min(|a|, |b|)}{\max(|a|, |b|)}. \tag{2.16}$$

This is the upper bound that Equation 2.2 already imposes on the target: two sequences of unequal length cannot exceed it whatever their contents. The baseline therefore measures how much of the internal embedding geometry mirroring sequence similarity can be explained by sequence length alone. It is applied unchanged to all four datasets and does not inspect symbol identities or their order.

Because all pairs with the same two lengths receive the same score, the baseline produces large groups of tied candidates. Spearman correlation assigns average ranks to tied scores, and AUROC retains its standard treatment of them. For MAP@10, the result is averaged over 200 random orderings within tied groups; the tie-handling procedure is specified in Section 2.6.

2.6 Evaluation Protocol

The evaluation distinguishes between pairwise similarity assessment and retrieval of neighbours for a query sequence. Pairwise evaluation measures how well a method orders or separates sequence pairs in comparison to their exact normalised Levenshtein similarity. Retrieval evaluation measures how well exact high-similarity neighbours can be recovered from the complete search collection.

Two evaluation objects are constructed for each dataset. First, a set of exact relevance sets, which determines which sequences are relevant to each retrieval query. Second, a decile-balanced pair set, which provides examples from across the similarity ranges of Table 2.1. All methods are evaluated using the same retained pairs, relevance sets and queries.

2.6.1 Exact Relevance Sets

Every sequence is compared with every other sequence in the same search collection using exact normalised Levenshtein similarity. Comparisons are performed only within the same dataset and representation. AA sequences are therefore compared only with AA sequences, SS with SS, 3Di with 3Di and synthetic strings with synthetic strings. For each sequence, its relevance set contains every other sequence whose exact normalised Levenshtein similarity is at least 0.70, corresponding to the high range of Table 2.1. This is an operational criterion on the normalised Levenshtein scale and is not to be interpreted as a biological threshold.

A sequence is used as a retrieval query only if this relevance set is non-empty. Only the members of each relevance set and their exact similarities are required for evaluation, not the full matrix of comparisons.

The number of eligible queries therefore differs sharply between datasets. Retrieval is performed as an exhaustive search over the complete collection of encoded sequences. No approximate nearest-neighbour index is built or evaluated at any point, so the reported retrieval quality is a property of the embedding rather than of a search structure.

2.6.2 Decile-Balanced Pairwise Evaluation Sets

The natural similarity distributions described in Section 2.4 and shown in Figure A.1 differ substantially between datasets. This difference is also visible in the number of high-similarity pairs found by the exhaustive comparison, shown in Table 2.6.

Table 2.6: High-similarity pairs in each search collection. The counts are exact and cover every pair of sequences within a collection.

Collection	Sequences	Pairs	Pairs with $s_{\mathrm{Lev}} \geq 0.70$
AA	10,501	55,130,250	5
3Di	10,501	55,130,250	6,009
SS	10,497	55,088,256	623,077

Sampling pairs in proportion to their natural frequency would therefore produce almost no high-similarity AA pairs but thousands of high-similarity SS pairs. The same metric would consequently focus on different parts of the similarity scale in each dataset. Balancing makes every similarity range comparably represented in every dataset, which allows for a fair spearman rank correlation across datasets.

For the three CATH-derived collections, 200,000 candidate pairs are selected by independently drawing two sequences from the corresponding collection. Every sequence has the same probability of being selected, and pairs containing the same sequence twice are discarded. For the synthetic evaluation dataset the candidate pairs are the 28,000 generated pairs themselves, since that dataset is constructed pairwise rather than as a collection of independent sequences. All high-similarity pairs identified by the exact relevance set comparison in Section 2.6.1 are then added to the candidate pairs, so that the high range is populated rather than left to chance sampling.

The candidate pairs are divided into ten intervals of equal width on the s_{Lev} scale. Interval k contains every pair with $k/10 \leq s_{\mathrm{Lev}} < (k+1)/10$ for $k = 0, \ldots, 8$, and the tenth interval additionally includes $s_{\mathrm{Lev}} = 1$. These intervals are fixed. The term *decile-balanced* is used throughout in that sense.

At most 400 pairs are retained from each occupied interval. If an interval contains fewer than 400 pairs, all available pairs are retained. The effect of this bound, why we choose 28,000 synthetic evaluation pairs, as well as the initial sample size of the protein-derived datasets are reported in detail in the Appendix B.

Adding the known high-similarity pairs and limiting the number of pairs per interval serve different purposes. Adding the known pairs prevents a sparsely populated high range from being missed during random sampling. Capping the number of pairs prevents a densely populated interval from dominating the pairwise evaluation metrics. A fully populated evaluation set therefore contains at most 4,000 pairs, with each interval contributing at most one tenth of them. Where the available pairs do not fill every interval the set is smaller and that even weighting is not reached.

The resulting pair sets are not samples of the natural pair distributions. They deliberately give comparable weight to different parts of the similarity scale. The reported pairwise metrics therefore describe performance across the balanced similarity range rather than performance under the natural frequency of sequence pairs.

2.6.3 Evaluation-Set Sizes

Table 2.7 reports the size of each evaluation set. The *decile-balanced pairs* are those remaining after the per-interval bound of Section 2.6.2 is applied. They are the set over which Spearman correlation and RMSE are computed. The *high-range pairs* are the subset with $s_{\mathrm{Lev}} \geq 0.70$, which form the positive class for AUROC. The *retrieval queries* are the sequences whose relevance set (Section 2.6.1) is non-empty, and are the queries over which MAP@10 is computed. A query is counted once per sequence rather than once per pair, so a sequence with many high-similarity partners contributes a single query.

Table 2.7: Sizes of the three evaluation sets defined in Section 2.6.

Dataset	Decile-balanced pairs	High-range pairs	Retrieval queries
Synth	3,648	1,205	2,410
3Di	3,699	1,225	347
SS	4,000	1,403	10,002
AA	1,216	5	10

The AA Spearman correlation is calculated from all $1{,}216$ pairs and does not depend only on the five high-range pairs. AA AUROC and high-range RMSE, in contrast, rest on five positive pairs, and AA MAP@10 on ten directed queries. These AA measurements are reported descriptively and are not treated as stable estimates of overall performance.

2.6.4 Spearman Rank Correlation

Spearman rank correlation measures whether a method preserves the ordering of exact normalised Levenshtein similarities within a dataset. For every retained pair, the similarity assigned by the evaluated method is ranked and compared with the rank of the pair's exact normalised Levenshtein similarity:

$$\rho = \frac{\sum_i \left(R_i - \bar{R}\right)\left(S_i - \bar{S}\right)}{\sqrt{\sum_i \left(R_i - \bar{R}\right)^2}\sqrt{\sum_i \left(S_i - \bar{S}\right)^2}}, \tag{2.17}$$

where R_i is the rank of the exact normalised Levenshtein similarity of pair i, S_i is the rank of the similarity assigned by the evaluated method, and $\bar{R}$ and $\bar{S}$ are the corresponding mean ranks. Tied values receive the average of the ranks they span, which is why the correlation is stated in this form rather than through the simplified expression that assumes distinct values.

A value of one indicates that the ordering is preserved exactly, a value of zero indicates no monotonic relationship, and a value of minus one indicates reversed ordering. Spearman correlation is reported over the complete decile-balanced set and separately within the far, mid and high ranges of Table 2.1.

2.6.5 AUROC for High-Similarity Discrimination

AUROC measures how reliably a method assigns a high score to a pair that is genuinely high-similarity. Write $v_i = s_{\mathrm{Lev}}(a_i, b_i)$ for the exact normalised Levenshtein similarity of pair i. Pairs reaching the high range are labelled as positives, and every remaining pair is labelled as negative, so that the negative class contains both far- and mid-range pairs (Table 2.1):

$$y_i = \begin{cases} 1, & v_i \geq 0.70, \\ 0, & v_i < 0.70. \end{cases} \tag{2.18}$$

AUROC can be interpreted as the probability that a randomly selected positive pair receives a higher score than a randomly selected negative pair. A value of 0.5 represents chance-level ordering, while a value of one represents complete separation, meaning that every high-similarity pair receives a higher score than every pair below the threshold.

2.6.6 MAP@10 for Retrieval

Mean average precision answers whether the high-similarity partners of a query are ranked early relative to less similar candidates, and whether they are retrieved consistently across queries. Every sequence with at least one exact high-similarity neighbour is used as a query, and the candidates are ordered by the similarity assigned by the evaluated method.

For a query q, let R_q be the number of sequences in its relevance set and let $\mathrm{rel}_q(k) \in \{0, 1\}$ indicate whether the candidate at rank k belongs to that relevance set. Precision at rank k is the proportion of relevant sequences among the first k retrieved results:

$$P@k(q) = \frac{1}{k} \sum_{j=1}^{k} \mathrm{rel}_q(j). \tag{2.19}$$

Average precision at rank ten averages this quantity over the ranks at which a relevant sequence actually occurs:

$$\mathrm{AP@10}(q) = \frac{1}{\min(R_q, 10)} \sum_{k=1}^{10} \mathrm{rel}_q(k)\, P@k(q). \tag{2.20}$$

The normalisation by $\min(R_q, 10)$ accounts for queries with fewer than ten relevant sequences: a query with three relevant neighbours can reach a value of one by retrieving all three first, and a query with more than ten relevant neighbours can reach a value of one if the first ten results are all relevant. MAP@10 is the arithmetic mean of Equation 2.20 over all eligible queries $q \in Q$:

$$\mathrm{MAP@10} = \frac{1}{|Q|} \sum_{q \in Q} \mathrm{AP@10}(q). \tag{2.21}$$

MAP@10 therefore measures both whether relevant neighbours appear among the first ten results and how early they occur.

Candidates that receive identical scores

A method does not always assign distinct scores to distinct candidates, and where it does not, the order of the affected candidates is undetermined. The length-ratio baseline and the Dice coefficient both produce such groups systematically; the similarities produced by SNNEED and by ESM-2 do not.

Where the ranking within a tied group is undetermined, MAP@10 is not a single number: it depends on an ordering the method does not supply. The value reported here is therefore the mean over 200 independent random orderings within tied groups, which estimates the MAP@10 that would be obtained on average if ties were broken at random. Precision, average precision and the retrieval collection itself are unaffected; only the arrangement inside a tied group is resampled.

2.6.7 RMSE for Numerical Accuracy

SNNEED is additionally evaluated on the numerical scale on which it was trained. Let

$$\mathcal{H} = \{ i \mid v_i \geq 0.70 \} \tag{2.22}$$

denote the high-range pairs in the decile-balanced evaluation set. Their root mean squared error is

$$\mathrm{RMSE}_{\mathrm{high}} = \sqrt{\frac{1}{|\mathcal{H}|} \sum_{i \in \mathcal{H}} \left(\widehat{s}_\theta(a_i, b_i) - v_i \right)^2}. \tag{2.23}$$

RMSE is reported only for SNNEED, because its parameter-free Euclidean readout is trained directly to approximate s_{Lev} and is the only method whose output is on that scale.

2.7 Computational Environment

2.7.1 Software

All experiments were carried out in Python 3.13.15 in Google Colab. Table 2.8 lists the components on which the reported results depend, together with the role each of them fulfils.

Table 2.8: Software used, with the version recorded by the run and the role each component fulfils.

Component	Version	Role	Reference
Python	3.13.15	Language runtime	
PyTorch	2.11.0+cu128	Encoder implementation, training and inference	[27]
NumPy	2.1.3	Numerical arrays; mean average precision and root mean squared error	[28]
SciPy	1.16.3	Spearman correlation; sparse representation of the 3-gram sets	[29]
scikit-learn	1.6.1	AUROC	[30]
RapidFuzz	3.14.5	Exact Levenshtein distance	[23]
Transformers	5.15.0	Pretrained ESM-2 model and its tokeniser	[31]
Matplotlib	3.10.0	Figures	[32]
pandas	2.2.3	Tabular handling of results	[33]

The pretrained ESM-2 model of Section 2.5.1 was obtained through the Transformers library [31].

Two of the reported metrics are not taken from a library. Mean average precision and the root mean squared error of Equation 2.23 are computed directly from the retrieved ranks and the predicted similarities, following the definitions of Section 2.6. Spearman correlation and AUROC are computed with the implementations listed above, using their standard treatment of tied values.

2.7.2 Hardware

Training and inference were performed in Google Colab on a single NVIDIA Tesla T4 graphics processing unit with 16 GB of memory, using CUDA 12.8 on an x86-64 Linux host. Every encoder reported in this thesis was trained on that configuration, and all evaluation was carried out on it.

3 Results

Trained only on synthetic strings, SNNEED reproduces the normalised Levenshtein relationship closely, and frozen ESM-2 shows that a task-agnostic representation already carries part of that signal before any supervision. The more demanding question is how much of that carries over when the same frozen encoder meets input distributions it was never trained on. Much of it does, though retrieval marks the limit.

Four metrics examine this from different angles: Spearman rank correlation for global ordering, MAP@10 for neighbour retrieval, AUROC for separating highly similar pairs, and RMSE for numerical error. For the first three, Length and Dice are reported first, then ESM-2 as the task-agnostic control, then SNNEED read against them. RMSE is reported for SNNEED alone, because only its readout predicts the target scale directly. Each section closes on what its metric contributes to the two research questions of Section 1.5, restated below.

Q1 — **Signal and task-specific improvement.** A general, task-agnostic protein encoder already carries a local similarity signal. Does ESM-2 also carry a global similarity signal, and can a task-specific encoder trained directly on that target yield a stronger signal?

Q2 — **Abstraction and transfer.** Can SNNEED abstract beyond its training distribution? Trained on one data distribution and tested on another, did it learn transferable properties of the algorithm's output relationship, and how far do those properties survive, or did it learn merely regularities of the training data?

3.1 Spearman Rank Correlation

Figure 3.1 reports the Spearman rank correlation between each method's similarity score and the exact normalised Levenshtein similarity, computed over the decile-balanced pair set of each evaluation dataset as described in Section 2.6.4.

The highest coefficient on Synth is obtained by the Dice coefficient, at 0.983, followed by SNNEED at 0.925. On 3Di and SS, SNNEED obtains the highest coefficients, at 0.950 and 0.961. The corresponding ESM-2 coefficients are 0.689 and 0.876, while Dice reaches 0.789 and 0.679.

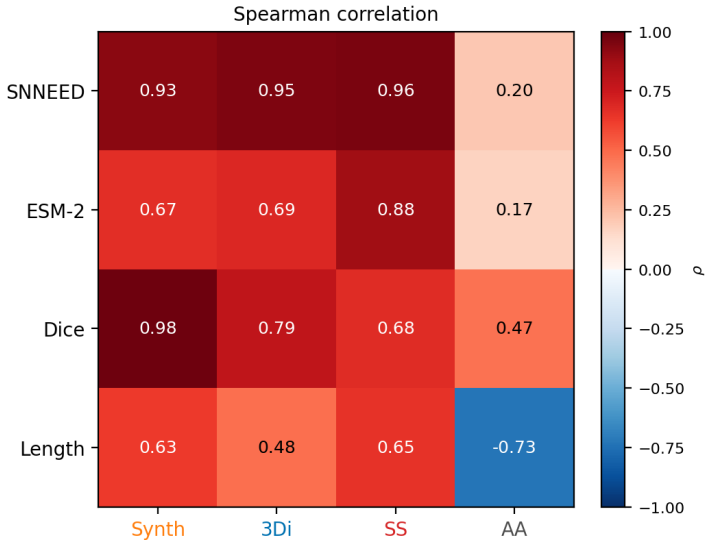


Figure 3.1: Spearman rank correlation between each method's similarity score and the whole range of exact normalised Levenshtein similarity scores.

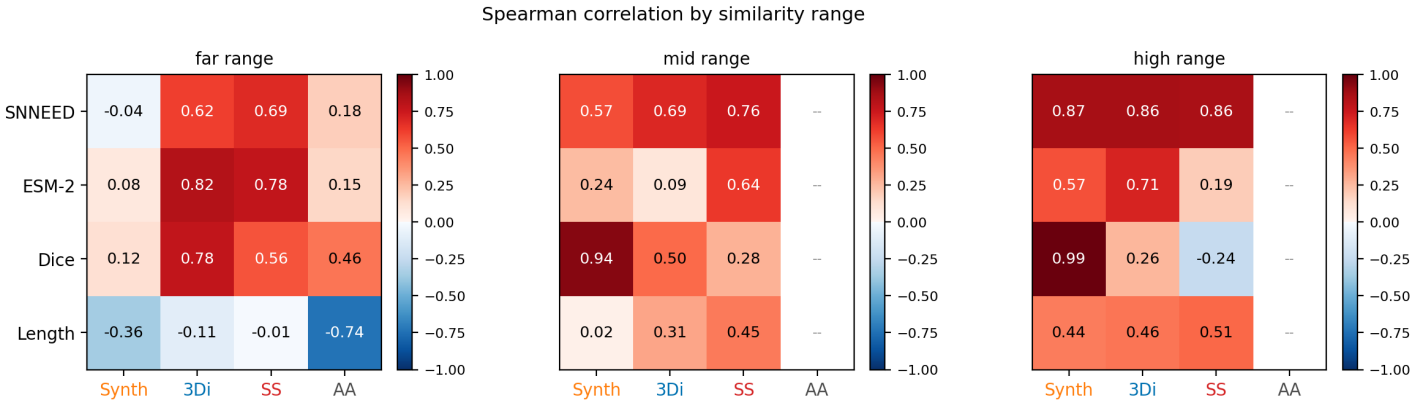


Figure 3.2: Spearman rank correlation between each method's similarity score and the exact normalised Levenshtein similarity, computed separately for the far, mid and high ranges of the target. The amino-acid column is blank in the mid and high ranges: 11 of its $1{,}216$ pairs fall in the mid range and 5 in the high range, too few for a rank correlation to be reported. Section 4.2.1 sets out why the collection supplies so few.

AA needs to be looked at separately, since it differs from the other three datasets. It yields the lowest coefficient of the four for every method, at 0.204 for SNNEED, 0.167 for ESM-2 and 0.474 for Dice, and it carries the only negative value, -0.732 for the length baseline. Because Spearman measures the monotone ranking relationship across the entire normalised Levenshtein similarity distribution, the coefficient depends on the distribution the dataset itself presents. Although the balancing procedure of Section 2.6.2 targets an equal number of pairs in every interval of that distribution, the amino-acid collection does not contain enough pairs for it to do so, as reported in Section B.1. The coefficient is nevertheless computed over $1{,}216$ pairs and is not limited by sample size.

The length baseline assigns a pair the ratio of the shorter to the longer sequence (Equation 2.16) and ignores the symbols entirely, so it establishes how much of the ordering follows from length alone. It attains overall correlations of 0.627, 0.481 and 0.655 on Synth, 3Di and SS, but its behaviour within the ranges is different: in the far range the correlations are -0.355, -0.107 and -0.013, whereas in the high range they are 0.441, 0.465 and 0.506. Much

of the whole-set association therefore follows from the contrast between ranges rather than from ordering within them, since pairs with a low length ratio cannot enter the high range, while pairs in that range necessarily have similar lengths. This confirms the bound of Equation 2.2 empirically. Within the deployed length interval the signal is faint, which is by design; Appendix C reports the measurements behind that interval.

The Dice coefficient compares the sets of distinct 3-grams occurring in two sequences (Equation 2.15) and establishes how well sequence pairs can be ordered on shared substrings alone, without any learned representation. It reaches 0.983 on Synth, the highest overall rank correlation obtained by any method in Figure 3.1, exceeding SNNEED's 0.925: near-perfect ordering on the in-distribution evaluation set is attainable by counting the 3-gram overlap of two sequences. On 3Di and SS the overall coefficients fall to 0.789 and 0.679, and the divergence between datasets is sharpest among the most similar pairs. In the high range, Dice attains 0.988 on Synth, 0.258 on 3Di and -0.243 on SS. The same comparison of substrings of length three therefore almost perfectly orders close Synth pairs, provides only a weak positive ordering on 3Di, and reverses the ordering on SS. That decline follows the symbol statistics of the three datasets, which Section 4.1 takes up.

Only $k = 3$ is evaluated here, so whether a larger window would restore the ordering on SS remains untested.

ESM-2 supplies the first learned representation, but it was not trained on normalised Levenshtein similarity. Similar to the approach in Fenoy et al. [15] discussed in Section 1.4, it is tested for whether a global similarity signal is already present in its frozen embedding geometry, by comparing its cosine similarity against the true normalised Levenshtein similarity. On Synth, 3Di and SS it reaches 0.669, 0.689 and 0.876. These datasets are representation-shift controls: their symbols are processed through the amino-acid tokeniser, so the coefficients describe whether a string-similarity signal survives that change, not ESM-2's biological capability on structural representations. Its overall AA coefficient of 0.167 reflects the skewed amino-acid scale described above. The overall coefficients conceal different behaviour across the scale. On Synth, the far-, mid- and high-range values are 0.075, 0.243 and 0.565. On 3Di they are 0.824, 0.092 and 0.708, a non-monotone pattern in which the mid range is ordered poorly. On SS the values decline from 0.781 in the far range to 0.637 in the mid range and 0.189 in the high range.

Figure 3.3 shows a further property of the frozen representation. On every dataset the cosine similarities saturate toward the top of their range, so that pairs of clearly different normalised Levenshtein similarity receive almost the same score. The effect is strongest on Synth, where the entire target scale is mapped into the interval above 0.94. A similar fingerprint has been reported for ESM-1b embeddings compared with BLASTp sequence identity, where cosine values likewise occupy only the upper end of their range across the entire identity axis (Figure 1.1) [15].

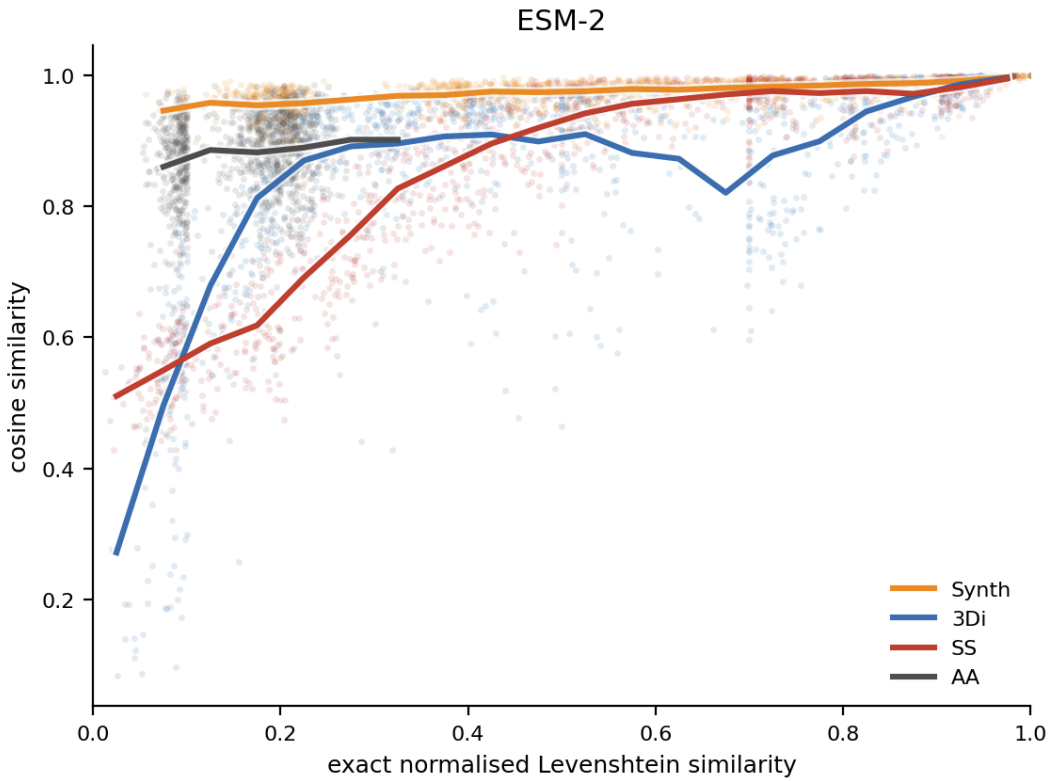


Figure 3.3: Cosine similarity of frozen ESM-2 embeddings plotted against exact normalised Levenshtein similarity, for all four evaluation datasets. Points are individual evaluation pairs and the solid lines are interval averages. Every curve flattens near the top of the cosine range, so that the scores of the most similar pairs are compressed into a narrow band; the compression is most pronounced on Synth, whose curve lies entirely above 0.94. The AA curve ends where its evaluation set does. The dip in the 3Di curve towards the upper end of the mid range corresponds to the non-monotone range pattern reported in the text.

Where ESM-2 carries the signal incidentally, SNNEED is supervised on it directly. It is trained on synthetically generated pairs with exact normalised Levenshtein similarity as the target and evaluated with frozen parameters on all four datasets. It attains overall correlations of 0.925, 0.950, 0.961 and 0.204 on Synth, 3Di, SS and AA, the highest of any method on 3Di and SS, and is exceeded only by its no-training baseline Dice on Synth. For Q1, direct supervision on the target therefore produces a higher overall coefficient than a task-agnostic, frozen ESM-2 embedding on every evaluation dataset: 0.925 against 0.669 on Synth, 0.950 against 0.689 on 3Di, 0.961 against 0.876 on SS, and 0.204 against 0.167 on AA. This advantage is not uniform across the scale. In the far range, ESM-2 exceeds SNNEED on Synth (0.075 against -0.040), 3Di (0.824 against 0.615) and SS (0.781 against 0.686).

For Q2, the high range gives the clearest rank-correlation evidence of transfer: 0.866 on Synth, 0.863 on 3Di and 0.859 on SS, three values differing by less than 0.01 although only Synth follows the training distribution. The two representations 3Di and SS pose a true test of whether SNNEED can transfer its learned behaviour outside what it has seen during training. Under the operational definition in Section 1.5, the ordering learned from synthetic pairs therefore transfers most clearly to close pairs in 3Di and SS. It does not establish recovery of the dynamic-programming procedure.

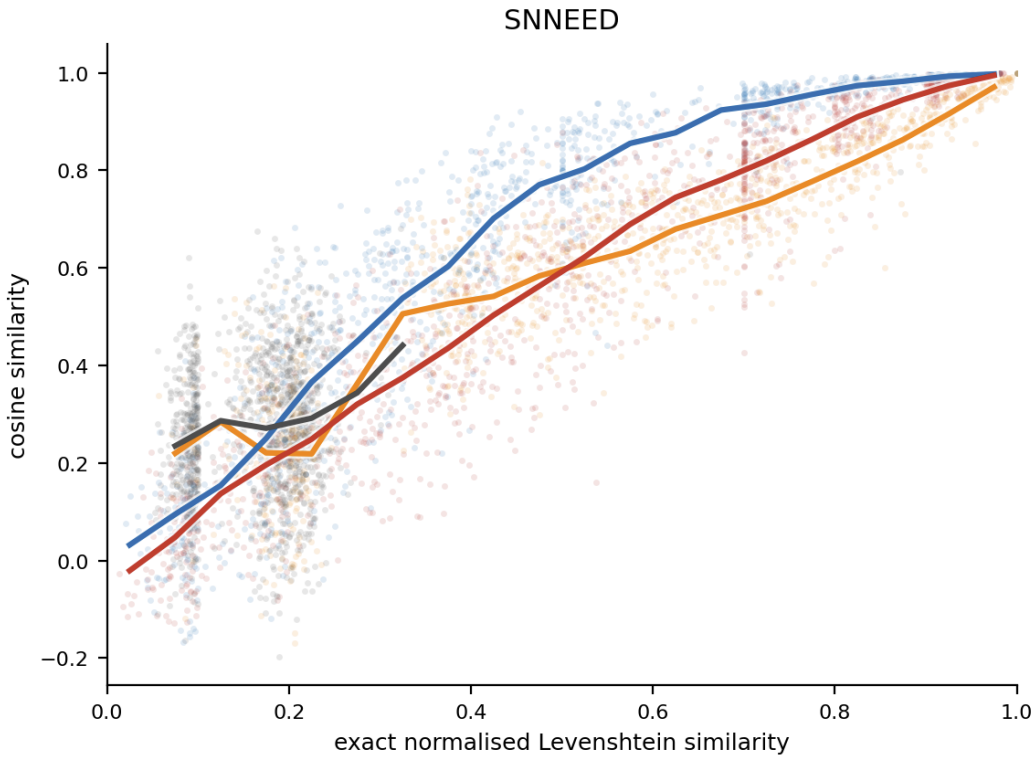


Figure 3.4: SNNEED's cosine similarity readout against true normalised Levenshtein similarity for each dataset. Describing a near diagonal line, indicating a strong correlation between the predicted similarity and the true similarity.

3.2 Mean Average Precision at Rank Ten

MAP@10 evaluates whether the exact high-similarity neighbours of a query occur among the first ten candidates returned by a method, with relevance defined as an exact normalised Levenshtein similarity to the query of at least 0.70 (Section 2.6.1). Unlike the Spearman evaluation, retrieval ranks each relevant neighbour against the complete search collection.

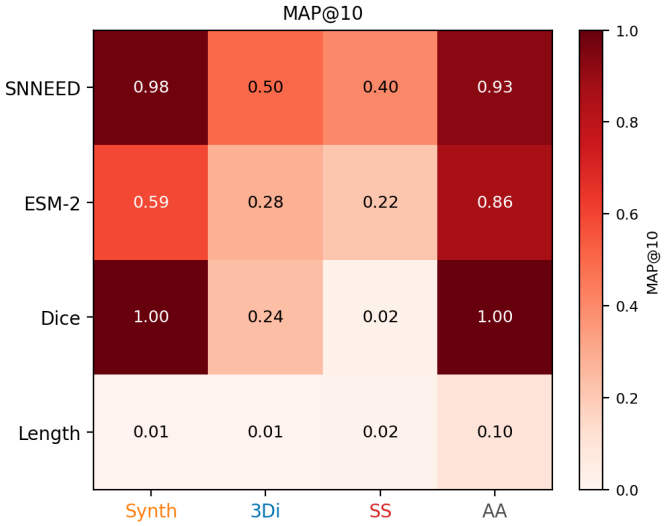


Figure 3.5: Mean average precision at rank ten for each method on each evaluation dataset. A candidate counts as relevant when its exact normalised Levenshtein similarity to the query is at least 0.70. The AA column rests on ten queries arising from five high-similarity pairs.

The amount of evidence differs substantially between datasets. The exact relevance sets, as described in Section 2.6.1, contain $1{,}205$ high-similarity Synth pairs, $6{,}009$ 3Di pairs and $623{,}077$ SS pairs, producing $2{,}410$, 347 and $10{,}002$ eligible queries, respectively. AA contains only five high-similarity pairs and therefore contributes ten queries. All AA MAP@10 values in this section are consequently descriptive and do not carry the cross-dataset conclusion.

Figure 3.5 reports the resulting MAP@10 values. The ordering of the methods differs across datasets. Dice obtains the maximum value of 1.000 on Synth, followed by SNNEED at 0.977. On 3Di and SS, SNNEED produces the highest values, at 0.502 and 0.405, followed by ESM-2 at 0.283 and 0.218. Dice falls to 0.239 on 3Di and 0.024 on SS. On the ten AA queries arising from five high-similarity pairs, Dice reaches 1.000, SNNEED 0.928, ESM-2 0.858, and Length 0.100.

The length baseline ranks candidates using only the ratio between the query length and the candidate length, which bounds normalised Levenshtein similarity from above but does not identify which similarly sized sequence is a relevant neighbour. It obtains MAP@10 values of 0.010, 0.008 and 0.016 on Synth, 3Di and SS, so relevant neighbours are almost never recovered near the beginning of the ranking when candidates are distinguished only by length. The positive whole-set correlations reported in Section 3.1 therefore do not translate into the recovery of particular high-similarity neighbours: the ordering between broad similarity ranges that produces them is insufficient for retrieval from the complete collection. This is the expected outcome within the deployed length interval, and confirms its purpose; Appendix C sets out how the interval was chosen to hold the length-ratio range narrow enough that length cannot serve as a shortcut to the target.

Dice reaches the maximum value of 1.000 on Synth, across $2{,}410$ queries. Retrieval adds a condition beyond the ordering examined in Section 3.1: related Synth pairs are generated by altering a shared base string, so they retain a common subset of 3-grams that separates them from the rest of the collection. That separation does not survive the change to the protein-derived representations. Dice falls to 0.239 on 3Di, where it ranks below both SNNEED and ESM-2, and to 0.024 on SS, only slightly above the length baseline's 0.016. The symbol statistics behind this contrast are taken up in Section 4.1. Dice also reaches 1.000 on AA, but this value is obtained from ten queries arising from five high-similarity pairs; it demonstrates perfect retrieval for these particular queries and is not a stable estimate of retrieval performance over the AA collection.

The comparison now turns from fixed rules to learned representations, beginning with ESM-2. Section 3.1 established that it carries a global similarity signal, and here it evaluates whether a task-agnostic representation can recover global high-similarity neighbours without direct supervision from normalised Levenshtein similarity. On Synth, frozen ESM-2 embeddings reach a MAP@10 of 0.588, substantially above the length baseline but below Dice and SNNEED. On 3Di and SS they reach 0.283 and 0.218, the second-highest retrieval result on both representation-shift datasets.

The comparison with Dice reverses across the datasets, with Dice stronger on Synth and ESM-2 stronger on both 3Di and SS: within the representation shifts, the frozen embedding retains more useful neighbour information than fixed 3-gram overlap. Its AA value of 0.858 is consistent with a useful retrieval signal in the representation for which ESM-2 was pretrained, but ten queries arising from five relevant pairs are too few to support a broader comparison.

These results describe the survival of a string-similarity signal and are not measurements of ESM-2's biological capability on 3Di or SS representations.

SNNEED is again the only method supervised on the target directly. For Q1, it produces stronger MAP@10 than frozen ESM-2 on every evaluation dataset: 0.977 against 0.588 on Synth, 0.502 against 0.283 on 3Di, 0.405 against 0.218 on SS, and, on the ten AA queries arising from five relevant pairs, 0.928 against 0.858. Task-specific supervision therefore improves the recovery of high-similarity neighbours relative to the task-agnostic representation, although the AA difference cannot support a general conclusion.

This advantage does not make SNNEED the strongest method in every setting. Dice reaches 1.000 on Synth, compared with SNNEED's 0.977, and also reaches 1.000 on the small AA query set, compared with 0.928. Section 4.1 takes this up as the expected outcome on an in-distribution set rather than a shortfall of the encoder. SNNEED's advantage appears on the two representation-shift datasets: it exceeds Dice by 0.263 on 3Di and by 0.381 on SS.

For Q2, the results show a meaningful transfer ability. SNNEED remains the strongest evaluated method on both 3Di and SS despite being trained exclusively on synthetic pairs. Its MAP@10 nevertheless decreases from 0.977 on Synth to 0.502 on 3Di and 0.405 on SS: the learned geometry preserves relative ordering while losing retrieval precision when relevant neighbours must be located within a much larger candidate set.

3.3 AUROC

AUROC measures whether a method separates sequence pairs that are highly similar from the remainder of the pairwise evaluation set. Pairs with an exact normalised Levenshtein similarity of at least 0.70 form the positive class, while the far- and mid-range pairs form the negative class as specified in Section 2.6.5. An AUROC of 1 indicates complete separation, whereas 0.5 indicates that the metric separates pairs no better than chance.

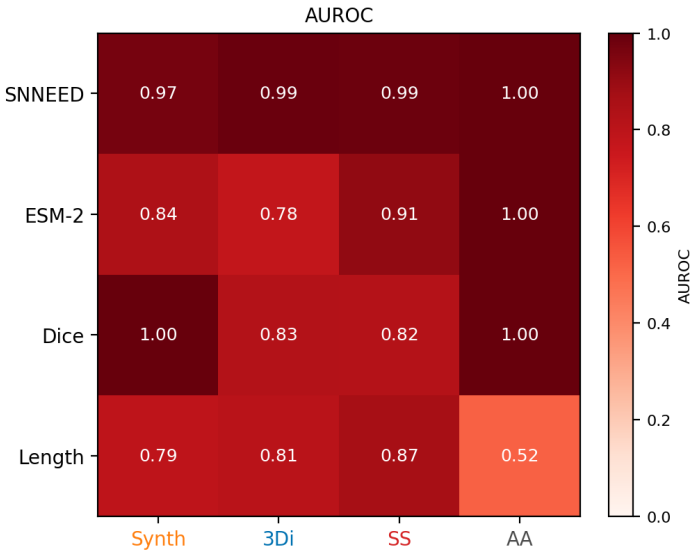


Figure 3.6: Area under the receiver operating characteristic curve for each method on each evaluation dataset. Pairs with an exact normalised Levenshtein similarity of at least 0.70 form the positive class and the far- and mid-range pairs the negative class. The AA column rests on five positive pairs against $1{,}211$ negative pairs.

The Synth, 3Di and SS evaluations contain $1{,}205$, $1{,}225$ and $1{,}403$ positive pairs, respectively. Their AUROC values are therefore supported across the high range. AA contains only five positive pairs against $1{,}211$ negative pairs. Every AA AUROC in this section is consequently descriptive and is not used for the cross-dataset conclusions.

On Synth, Dice produces the highest AUROC at 0.998, followed by SNNEED at 0.969. On 3Di and SS, SNNEED obtains the highest values, at 0.992 and 0.986. ESM-2 produces the second-highest result on SS, at 0.912, whereas Dice ranks second on 3Di at 0.825. Even the Length baseline reaches between 0.790 and 0.867 on these three datasets. High-similarity discrimination is therefore achieved more broadly than the retrieval performance reported in Section 3.2.

AUROC is invariant to the proportion of positive pairs. Changing that proportion without changing how the two classes are scored leaves the value unchanged, whereas precision-based measures do respond to it, which makes them more informative when relevant pairs are rare [34]. What AUROC does depend on is which pairs make up the negative class, since a far-range pair is far easier to separate from a high-range pair than a mid-range one is. The decile-balanced sets of Section 2.6.2 populate the intervals below the threshold evenly, so the negative class contains mid-range pairs rather than consisting almost entirely of far-range pairs as it would under natural sampling. AA is the exception. Its collection supplies no mid-range pairs at all (Appendix B). Its AUROC values are therefore reported descriptively.

Dice reaches an AUROC of 0.998 on Synth, the highest value obtained by any method on that dataset. Its near-perfect discrimination agrees with its MAP@10 of 1.000: under the controlled Synth distribution, shared 3-grams both separate high-similarity pairs from the remaining pairs and recover the relevant neighbours of individual queries.

On 3Di and SS, Dice falls to 0.825 and 0.821. These values remain well above chance, but they no longer approach the Synth result. The decline is consistent with the distribution sensitivity identified in Sections 3.1 and 3.2.

The SS result also illustrates the difference between discrimination and retrieval. Dice reaches an AUROC of 0.821 but a MAP@10 of only 0.024. Shared 3-grams therefore provide enough information to assign high-range pairs generally higher scores than far- and mid-range pairs, but not enough to identify the correct neighbours near the top of a complete search ranking. Noisy class separation does not imply useful retrieval.

Turning again to the learned representations, ESM-2 obtains AUROC values of 0.841, 0.777 and 0.912 on Synth, 3Di and SS. The frozen representation therefore separates high-similarity pairs from the remaining pairs without direct supervision from normalised Levenshtein similarity. Its behaviour varies between datasets. On SS, ESM-2 produces the strongest reference-method result, exceeding Length at 0.867 and Dice at 0.821. On 3Di, however, its value of 0.777 is lower than both Dice at 0.825 and Length at 0.806. The task-agnostic similarity signal is therefore not uniform across representations.

As with Dice, ESM-2's discrimination is stronger than its retrieval performance. On SS it reaches an AUROC of 0.912 but a MAP@10 of 0.218. On 3Di the corresponding values are 0.777 and 0.283. The frozen geometry can broadly distinguish close from non-close pairs without consistently placing the exact relevant neighbours among the first ten candidates.

SNNEED is again supervised on the target directly. For Q1, that supervision strengthens high-similarity discrimination relative to frozen ESM-2. SNNEED exceeds ESM-2 on Synth, at 0.969 against 0.841. On 3Di, at 0.992 against 0.777, and on SS, at 0.986 against 0.912.

SNNEED is not the strongest method on every dataset. Dice reaches 0.998 on Synth and therefore exceeds SNNEED's 0.969 by a small margin. The ordering reverses after the representation shift: SNNEED reaches 0.992 and 0.986 on 3Di and SS, while Dice falls to 0.825 and 0.821. The same direct supervision that does not improve upon Dice under the favourable Synth distribution produces substantially more stable discrimination when the sequence representation changes.

For Q2, AUROC provides the strongest evidence of transfer among the reported metrics. SNNEED's values on 3Di and SS are not lower than its Synth value. Instead, they remain close to complete separation. A high-similarity pair is therefore almost always scored above a far- or mid-range pair even after the encoder leaves its training distribution.

Taken together, the AUROC results show that high-similarity discrimination is easier than high-similarity retrieval. Length, Dice and ESM-2 all achieve substantial AUROC values in settings where their MAP@10 is low, demonstrating that broad class separation alone is not evidence for useful nearest-neighbour retrieval. SNNEED stands out by combining near-complete discrimination on 3Di and SS with the highest MAP@10 on both datasets. Under this metric, direct, task-specific supervision produces a high-similarity signal that transfers strongly.

3.4 Root Mean Squared Error

This metric evaluates how well SNNEED's predicted values are numerically close to its corresponding exact normalised Levenshtein similarities. Unlike Spearman correlation, AUROC and MAP@10, RMSE is reported only for SNNEED because its Euclidean-distance readout is trained specifically to approximate the normalised Levenshtein similarity scale.

The evaluation is restricted to the high range, $s_{\mathrm{Lev}} \geq 0.70$, because numerical accuracy remains most useful for finding and evaluating sequence pairs of high similarity. RMSE ranges from 0 to 1. A value of 0 indicates that every predicted similarity equals its exact value, while larger values indicate greater numerical error. Lower RMSE is therefore better.

Table 3.1: Root mean squared error between SNNEED's predicted similarity and the exact normalised Levenshtein similarity, computed over the high-range pairs of each evaluation dataset. The AA value rests on five pairs and is reported for completeness only. RMSE is reported for the high range alone, where a numerically accurate prediction is actionable; the mid and far ranges are not computed.

Dataset	High-range pairs	RMSE
Synth	1,205	0.113
3Di	1,225	0.070
SS	1,403	0.054
AA	5	0.108

SNNEED obtains an RMSE of 0.113 on Synth in the high range. The error decreases even more after the transfer test, reaching 0.070 on 3Di and 0.054 on SS. These values are supported by $1{,}225$ and $1{,}403$ high-range pairs, respectively.

The lower RMSE on 3Di and SS shows that the numerical meaning of SNNEED's Euclidean-distance readout survives the representation shift. The transfer results are therefore not limited to preserving an ordering: on the high-range pairs, the predicted values also remain close to the exact normalised Levenshtein similarities on the scale on which the model was trained. In particular, the SS result of 0.054 corresponds to the smallest numerical error despite SS having the smallest alphabet and the most substantial change in local sequence statistics. On AA, SNNEED obtains an RMSE of 0.108, but this value is calculated from only five high-range pairs. Its similarity to the Synth result cannot support a general conclusion about numerical accuracy on AA and is reported only for completeness.

The result establishes that numerical accuracy does not deteriorate under representation shifts. RMSE consequently strengthens the evidence for Q2: the task-specific input-output mapping learned from synthetic pairs remains useful on 3Di and SS without retraining.

3.5 Conclusion

Taken together, the results answer Q1 positively: frozen ESM-2 embeddings contain a measurable global similarity signal. Direct, task-specific supervision on normalised Levenshtein similarity strengthens this signal under overall rank correlation, high-similarity discrimination and retrieval.

Q2 is also supported but with qualifications. Trained only on synthetic strings, SNNEED retains strong rank correlation, near-complete high-similarity discrimination, low numerical error and the highest MAP@10 on 3Di and SS. However, MAP@10 falls from its in-distribution result 0.977 on Synth to 0.502 and 0.405, showing that the complete neighbourhood structure is not perfectly preserved. The results therefore demonstrate transfer of a useful approximation beyond the training distribution.

4 Discussion

Chapter 3 reported what each method achieves on each representation. This chapter asks why those values came out as they did. It follows the two questions in the order they were posed: what direct supervision on normalised Levenshtein similarity buys over a task-agnostic representation, and what survives when the trained encoder is applied to representations it never saw. Both answers turn on the statistics of the inputs, so the chapter then establishes the chance floor those statistics impose on the similarity scale itself, examines the architecture and training decisions behind the reported configuration, and closes on the limits of the approach and the directions it leaves open.

4.1 The Value of Task-Specific Supervision

The comparison between ESM-2, SNNEED and Dice reveals a clear tension. SNNEED achieves a higher overall Spearman rank correlation than frozen ESM-2 on all four evaluation datasets, indicating that direct supervision strengthens the relationship with the similarity target. On Synth, however, Dice reaches 0.983 and exceeds SNNEED's 0.925 without requiring training. The benefit of task-specific supervision is therefore not obvious at first glance. Where it lies is what this section sets out, and Section 4.2 then asks how much of it survives outside the training distribution.

Direct supervision does not create the similarity signal from nothing, since frozen ESM-2 already contains a measurable version of it. Its frozen representations preserve a global similarity signal, but Figure 3.3 shows that this signal flattens near the top of their scale. Pairs with substantially different normalised Levenshtein similarities can consequently receive similar cosine scores. This compression still permits ESM-2 to distinguish broad parts of the target range, but provides less resolution between pairs whose similarities are close. Its pre-training objective never rewarded the preservation of global edit similarity, so the relationship emerges as a by-product of a task-agnostic representation.

SNNEED, on the other hand, receives a specific training signal. Each training pair contributes its exact normalised Levenshtein similarity to the mean squared error objective (Section 2.3.2), which directly encourages different target values to be represented by different Euclidean distances. What that supervision can sharpen is limited by what the training pairs cover. The

generator of Section 2.3.1 distributes its symbols independently and uniformly, so a training pair carries as little sequence pattern beyond the edit relationship itself as the 20-symbol alphabet allows, and the encoder is left with the edit relationship as the only regularity available to it.

The same construction, however, limits the range of normalised Levenshtein similarities over which supervision is given. Altered copies rarely obtain similarities below 0.30, and even independently generated strings over the 20-symbol alphabet reach a median of only 0.183 (Section 4.3). The low end of the target range is therefore thinly populated in training, because pairs that dissimilar are difficult to construct over this alphabet at all. The far-range coefficients reflect this: there ESM-2 exceeds SNNEED on Synth, at 0.075 against -0.040. Task-specific supervision sharpens the distinctions its training data represents, and the distinctions at the bottom of the scale are largely absent from it.

The Dice result on Synth provides a complementary boundary. Synthetic symbols are sampled independently and uniformly from a 20-symbol alphabet, giving $20^3 = 8{,}000$ possible 3-grams, so accidental overlap between unrelated synthetic strings is limited. The related strings of a generated pair, in contrast, descend from the same base string and retain many of its local substrings after editing. Shared 3-grams are therefore particularly informative under this generator, and Synth is the case a comparison of 3-grams is best suited to. That a fixed rule orders these pairs more accurately than a trained encoder is the expected outcome rather than a shortfall of the encoder. Task-specific learning is not required where a fixed statistic already captures the regularity connecting the inputs to the target. The Dice coefficient is itself an algorithm in the sense of Chapter 1: a fixed set of rules whose behaviour is determined in advance and whose result is dependable for as long as its inputs satisfy the assumptions behind it.

How far that regularity extends can be read from the symbol statistics of Section 2.4.3. Effective alphabet size and first-order mutual information order the four datasets identically: Synth at 20.00 symbols and 0.000 bits, AA at 17.91 and 0.007, 3Di at 13.93 and 0.551, and SS at 2.88 and 0.875 (Tables 2.4 and 2.5). Both quantities are first order and describe only the dependence between adjacent positions. Synth and AA sit at one end, their symbols close to uniform over their own alphabet and almost independent of the symbol preceding them. SS sits at the other, where more than half of a position's character entropy is already determined by its predecessor. The three protein-derived datasets encode the same domains and therefore share one length distribution, so it is these statistics that separate them.

Dice's results follow those statistics. Its high-range ordering falls from (0.988) on Synth to (0.258) on 3Di and reverses to (-0.243) on SS, as the symbols become less uniform and more dependent on the one before them. Dice's rules hold while their assumptions hold and degrade as the input moves away from them. On AA, whose statistics sit beside Synth's, Dice again exceeds SNNEED in overall rank correlation, at (0.474) against (0.204) over (1,216) pairs, which is what its statistics predict.

Task-specific supervision therefore adds resolution rather than signal. Frozen ESM-2 already carries a measurable global similarity relationship but compresses it toward the top of its scale, and supervision separates the pairs that compression leaves indistinguishable; Dice's reversal on SS shows that a fixed comparison of 3-grams cannot supply that resolution once the symbol statistics change.

This returns to the motivation of Chapter 1, whether the generalisation potential of a learned representation can be combined with the specificity of an algorithmically defined target. SNNEED shows that it can, holding its ordering on the representations where the assumptions behind a fixed comparison no longer hold.

4.2 Transfer Across Representations

The transfer results become clearer when the evaluation metrics are not treated as interchangeable measures of the same property. Spearman correlation, AUROC, MAP@10 and RMSE place different demands on the learned representation. SNNEED preserves some of these properties almost unchanged outside its training distribution, while others deteriorate more visibly. The contrast between them identifies both the extent and the limits of the transferred approximation.

The clearest continuity appears among highly similar pairs. SNNEED obtains high-range Spearman correlations of 0.866 on Synth, 0.863 on 3Di and 0.859 on SS. These values differ by less than 0.01 even though the model is trained only on synthetic strings. The ordering of close pairs therefore remains stable when the input changes from uniformly generated amino-acid symbols to the structured 3Di and secondary-structure representations. This is more specific than the strong overall correlations on 3Di and SS, because it shows that the ordering is preserved in the part of the similarity range most relevant to retrieval.

High-similarity discrimination transfers more strongly still. SNNEED reaches AUROC values of 0.992 on 3Di and 0.986 on SS, compared with 0.969 on Synth. A high-similarity pair is therefore almost always scored above a randomly selected pair from the far or middle range, even after the representation changes. Under this requirement, leaving the synthetic training distribution does not produce a visible deterioration. The model preserves the broad separation between highly similar and less similar pairs.

The numerical meaning of the readout also remains useful. SNNEED's high-range RMSE decreases from 0.113 on Synth to 0.070 on 3Di and 0.054 on SS. Its predictions on the two transferred representations therefore remain close to the exact normalised Levenshtein similarities on the scale used during training.

Retrieval presents a different picture. SNNEED's MAP@10 declines from 0.977 on Synth to 0.502 on 3Di and 0.405 on SS. It remains the strongest evaluated method on both transferred representations, but it does not recover their relevant neighbourhoods within the first ten positions as consistently as it recovers the generated partner of its in-distribution dataset. The transfer is therefore useful but incomplete under the most demanding ranking criterion.

This difference does not contradict the high AUROC values. AUROC averages over comparisons between positive and negative pairs. A method can perform well on this broad comparison even when some non-relevant candidates receive scores close to those of the true neighbours. MAP@10 examines a narrower and more demanding part of the ranking: it asks whether relevant candidates occupy the first ten positions for each individual query. A relatively small number of highly scored false candidates may have little effect on the average positive–negative comparison measured by AUROC, while still displacing relevant neighbours from the top ten.

Each dataset also presents a different neighbourhood behaviour. Synth contains $1{,}205$ high-similarity pairs and $2{,}410$ eligible queries, so every query has exactly one relevant partner and its average precision is determined by the rank of that single sequence. The $6{,}009$ high-similarity 3Di pairs provide an average of 34.6 relevant neighbours across 347 queries, and the $623{,}077$ SS pairs an average of 124.6 across $10{,}002$. The collections searched are comparable in size, at $7{,}296$ sequences for Synth, $10{,}501$ for 3Di and $10{,}497$ for SS, so the difference lies in how many relevant neighbours a query has rather than in how much there is to search. For queries with at least ten relevant neighbours, a perfect MAP@10 contribution requires all of the first ten positions to be relevant.

These differences mean that MAP@10 does not pose an identical retrieval problem on every dataset. Synth asks the model to locate one generated partner, while 3Di and SS ask it to order neighbourhoods containing many valid candidates. Having more relevant candidates may provide more opportunities to fill the top ten, but it also changes what a perfect ranking requires. This alone does not establish why MAP@10 declines. Comparisons between methods within the same dataset remain unaffected, because every method is evaluated against the same queries and candidates.

Taken together, the metrics indicate that SNNEED transfers the broad geometry of the target more reliably than the detailed neighbourhood structure. It continues to order close pairs, separate them from pairs of lower similarity and approximate their numerical target values. It is less successful at preserving the exact ordering required to populate the first ten positions of every query ranking. The three views therefore come apart, which is why Section 1.5 treats them as complementary rather than interchangeable: a method can separate close pairs from distant ones without completely recovering the right neighbour, the distinction Chapter 1 draws between a similarity signal being present and being resolved finely enough to discriminate.

All in all, SNNEED was never exposed to biological sequences during training, yet its representation remains useful on 3Di and SS, so its behaviour is not restricted to the exact symbol statistics of the synthetic generator. This does not demonstrate that the network recovered the procedure of the Levenshtein algorithm. What the results support is transfer of the learned input–output relationship of the Levenshtein algorithm itself.

4.2.1 The Amino-Acid Case

The amino-acid evaluation cannot be interpreted in the same way as the 3Di and SS evaluations. As Section 4.1 noted, AA's symbol statistics sit beside Synth's, and the ordering among the methods follows them: Dice, whose comparison depends on uniform and largely independent symbols, again comes first at 0.474. What does not follow those statistics is the level every method reaches. Length produces an overall Spearman correlation of -0.732, SNNEED 0.204 and ESM-2 0.167, and all four depart from their behaviour on Synth, 3Di and SS.

That departure is a property of the collection rather than of the methods. CATH-S20 is redundancy-reduced (Section 2.4.1), so pairs with high global amino-acid similarity are deliberately rare, and the decile-balanced evaluation set has a median normalised Levenshtein similarity of 0.181. Independently generated strings over the same 20-symbol

alphabet reach a median of 0.183 (Section 4.3). The evaluated pairs therefore sit essentially at the similarity expected of unrelated strings. The methods are being compared almost entirely within a narrow, low-similarity region in which small score differences determine the ordering, and no balancing of the evaluation set can supply pairs the collection does not contain.

The overall AA Spearman result is not limited by sample size. It is calculated from $1{,}216$ pairs and is therefore supported by substantially more observations than the high-range evaluations. Its limitation is the restricted target range. Only five AA pairs reach the threshold of 0.70, producing ten eligible retrieval queries, so the resulting AUROC, RMSE and MAP@10 values describe those particular pairs and do not support a general comparison across the amino-acid collection.

AA therefore serves as a limited control rather than a fully supported comparison of the methods across the complete similarity range. It demonstrates how the evaluated methods behave when almost the entire collection lies near the similarity expected from unrelated strings. It cannot establish whether the transferred encoder would preserve ordering or retrieval across a well-populated range of natural amino-acid similarities.

4.3 Chance Floor

Normalised Levenshtein similarity has a range of $[0, 1]$, but unrelated strings do not distribute uniformly across this interval. Strings drawn independently from a finite alphabet share certain similarities by chance alone, and a global alignment can track these accidental overlaps. Consequently, the lower end of the practically observed score distribution is guided by an alphabet-, composition- and length-dependent statistical chance floor.

A theoretical analogy is provided by the longest common subsequence (LCS) problem, which asks for the longest subsequence common to a set of sequences, most often just two. Let $L_n^{(q)}$ be the LCS length of two independently and uniformly drawn strings of length n over an alphabet of size q. Let $\mathbb{E}$ denote its expected value over the random draw of both strings. The average share of the string length that two unrelated strings can be expected to hold in common has been denoted as γ_q. For long strings their longest common subsequence occupies about $\gamma_q n$ symbols. Chvátal and Sankoff showed that this average is well defined, in the sense that the limit

$$\gamma_q = \lim_{n\to\infty} \frac{\mathbb{E}\left[L_n^{(q)}\right]}{n}$$

exists [35], while its exact value is not known for general q. Even two unrelated random strings therefore share a common subsequence whose expected length grows proportionally with the strings themselves. How that average shifts as the alphabet grows is a separate question, answered by a decay rate: Kiwi, Loebl and Matoušek proved that $\gamma_q\sqrt{q}$ tends to 2 for large q [36].So for large alphabets the shared fraction falls approximately as $2/\sqrt{q}$, signifying a decay rate as alphabet size increases. The similarity of unrelated strings is therefore not arbitrary. Its expected value follows from the sequence length and the alphabet size, even if the constant of proportionality is not known exactly.

This does not provide an exact chance floor prognosis for the normalised Levenshtein similarity used in this thesis. LCS permits only insertion and deletion, whereas unit-cost Levenshtein distance also permits substitution. The LCS result however motivates an expectation of a non-zero, alphabet-dependent chance baseline, since both are related. For two strings of equal length n, with LCS length L, Levenshtein distance d_{Lev} and normalised similarity $s_{\mathrm{Lev}} = 1 - d_{\mathrm{Lev}}/n$, the quantities satisfy

$$n - L \;\le\; d_{\mathrm{Lev}} \;\le\; 2(n - L), \qquad \text{and therefore} \qquad 2\frac{L}{n} - 1 \;\le\; s_{\mathrm{Lev}} \;\le\; \frac{L}{n}.$$

The relation between LCS length and Levenshtein similarity is therefore tight among highly similar strings and loosens when the strings are less similar.

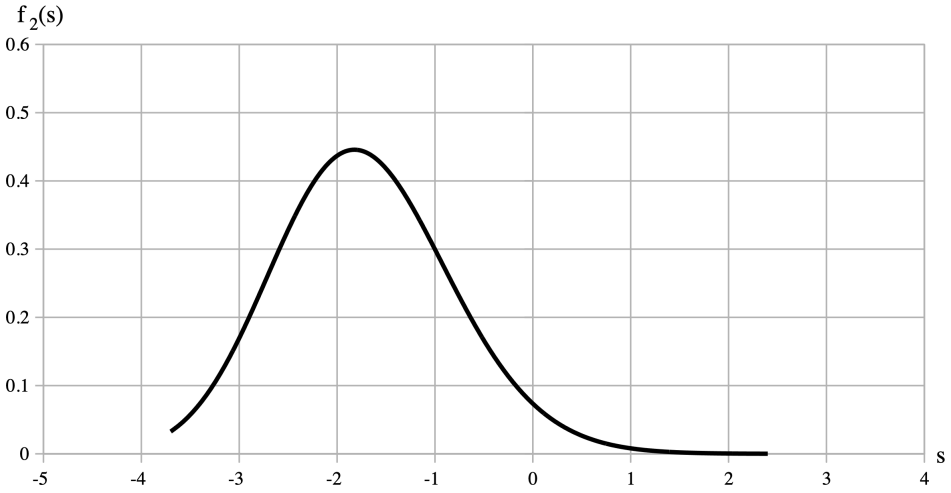


Figure 4.1: Density of the Tracy–Widom distribution for the Gaussian unitary ensemble, the case arising in the Bernoulli matching model; the subscript of f_2 identifies that ensemble. The horizontal axis is the centred and scaled longest-common-subsequence length s and the vertical axis its probability density. The distribution is asymmetric, with a short left tail and a longer right tail. Adapted from a figure by Ignatus, released under CC0 via Wikimedia Commons.[1]

Further, LCS gives a theoretical motivation for treating random-string alignment scores as a structured distribution. The longest common subsequence (LCS) problem is in a simplified variant of the problem, the Bernoulli matching model, in which potential symbol matches are treated as independent, the LCS length has been shown to follow, after centring and scaling, the Tracy–Widom distribution of the largest eigenvalue of a random matrix from the Gaussian unitary ensemble (Figure 4.1) [37, 38]. The result holds for that simplified model and not for the longest common subsequence of random strings itself, which remains open. It is quoted here only as motivation for expecting the scores of unrelated strings to concentrate in a narrow band rather than spreading across the range. Where that band sits is a separate matter, and it is set by the alphabet: the decay of γ_q above fixes the location, the Tracy–Widom result only the concentration.

The chance floors relevant to the four datasets used in this work are measured directly, and the score distributions of Appendix D do show that concentration. Over the 20-symbol alphabet shared by Synth, 3Di and AA, independently generated strings obtain a median similarity of 0.183, with 95 % of pairs between 0.119 and 0.222; they never reach 0.3. The floor is a property of the alphabet rather than of the representation: all three datasets agree to within 0.001. Secondary structure, drawn from three symbols, has a far higher floor at a

median of 0.520, with 95 % of pairs between 0.328 and 0.586. The direction of that difference is what the LCS result leads one to expect: with fewer symbols, accidental agreement is more likely, so the expected shared fraction γ_q rises as the alphabet shrinks. The comparison remains qualitative, since the floor is measured on normalised Levenshtein similarity rather than on the longest common subsequence. The shuffled null of Appendix D, which preserves the composition of the real sequences, moves each floor by a small amount and not always in the same direction, to 0.197 for AA, 0.236 for 3Di and 0.471 for secondary structure, so composition contributes, while the difference between alphabets remains much the larger effect.

The chance floor also explains the composition of the Synth evaluation set. Altered copies generated for training rarely obtain similarities below 0.30. Without independently generated pairs, the two lowest similarity deciles are empty. Even among $8{,}000$ independent pairs, only 48 obtain a score below 0.10. Increasing this number to $20{,}000$ yields only 107 such pairs and changes the balanced evaluation-set size from $3{,}648$ to $3{,}707$. The scarcity of scores below 0.10 is therefore a property of random strings over the 20-symbol alphabet rather than merely insufficient sampling.

4.4 Architecture and Training Decisions

The transfer behaviour reported in Chapter 3, and the accuracy with which the predicted values match the true normalised Levenshtein similarities, are conditional on configuration choices that Chapter 2 describes. This section examines those decisions, namely the effect of pooling and of the pooling width, the number of training epochs, and the size of the training set itself. Different configurations were evaluated on the same fixed evaluation sets as Chapter 3. The deployed model, which uses pooling with $K = 16$, 30 epochs and $30{,}000$ training pairs, serves as the reference configuration. The values reported for it below are those of Chapter 3, which the present run reproduces to within 0.007 on Synth, 3Di and SS.

4.4.1 Effect of the Pooling Width

The number of pooled positions in a vectorised sequence is set by the value of K. Increasing K preserves more positional resolution but also increases the input to the final projection layer, where each pooled position accounts for $8{,}192$ parameters. The three configurations therefore contain $75{,}648$, $141{,}184$ and $272{,}256$ parameters for $K = 8$, $K = 16$ and $K = 32$, respectively. So pooling width contributes to the final overall size of the encoder itself.

Overall Spearman correlation becomes worse as the pooling width increases, and it does so on all four datasets (Table 4.1). The decline is smallest on Synth, from 0.932 to 0.910, and largest on amino acids, from 0.318 to 0.142.

Retrieval shows a different relationship with pooling width. Synth MAP@10 decreases from 0.984 at $K = 8$ to 0.977 at $K = 16$ and 0.946 at $K = 32$, whereas 3Di rises from 0.462 to 0.502 and 0.528, and SS from 0.314 to 0.405 and 0.435. The SS figure is the clearest single

Table 4.1: Spearman rank correlation and mean average precision at rank ten for three pooling widths and for the configuration in which pooling is removed and the convolutional output flattened. The reference row reports the values of Chapter 3 at K=16. The AA retrieval values rest on ten queries with five relevant pairs among 1,211 negatives.

		Spearman				MAP@10			
Configuration	Parameters	Synth	3Di	SS	AA	Synth	3Di	SS	AA
$K = 8$	75,648	0.932	0.959	0.967	0.318	0.984	0.462	0.314	1.000
$K = 16$ (deployed)	141,184	0.925	0.950	0.961	0.204	0.977	0.502	0.405	0.928
$K = 32$	272,256	0.910	0.928	0.932	0.142	0.946	0.528	0.435	0.714
no pooling	1,648,512	0.847	0.843	0.802	−0.100	0.724	0.452	0.315	0.387

effect in the sweep: narrowing the pooling to $K = 8$ costs more than a fifth of the relevant neighbours that the deployed configuration recovers.

Pooling width therefore trades retrieval on Synth and amino-acid sequences against retrieval on 3Di and secondary structure. Narrower pooling ranks the relevant neighbours higher on the first pair, whereas wider pooling recovers more of them on the second. The grouping does not follow the distinction between the representation the encoder was trained on and the representations it is transferred to: amino-acid sequences are a transferred representation, yet their rank correlation falls with K exactly as Synth's does, from 0.318 at $K = 8$ to 0.204 at $K = 16$ and 0.142 at $K = 32$. For this architecture, $K = 16$ is a middle ground between the two groups.

4.4.2 Effect of Removing Pooling

The role of the operation becomes clearer when it is removed altogether. The projection layer requires a fixed-size input, and pooling is what converts a feature map of varying length into one. Without it, the only option that retains every position is to flatten the convolutional output at its full padded width of 200 positions rather than the 16 the deployed configuration pools to. The parameter count rises from $141{,}184$ to $1{,}648{,}512$, a factor of 11.7, and the final training error falls by a factor of 5.5, from 0.00210 to 0.00038. Despite this closer fit to the training pairs, performance deteriorates under every reported metric on every dataset (Table 4.1). The same holds for the two measures the table omits: AUROC falls on all four datasets and the error on high-similarity pairs rises on all four. Rank correlation on amino acids becomes negative, at -0.100. Why the amino-acid correlation changes sign rather than merely weakening is not established here, and is taken up in Section 4.6.

The comparison is not clean, in that removing pooling raises the parameter count by a factor of 11.7 at the same time, so capacity and pooling change together. The pooling-width results constrain the alternative reading. Widening the pooling from $141{,}184$ to $272{,}256$ parameters improved retrieval on 3Di and SS; if capacity alone determined the outcome, the far larger increase to $1{,}648{,}512$ should have pointed the same way. It points the opposite way on every dataset and every metric. This does not isolate pooling, since the second increase is an order of magnitude larger and capacity effects need not be monotone, but the deterioration is not explained by capacity alone.

4.4.3 Effect of the Epoch Budget

SNNEED's performance after 5, 10, 20, 30 and 50 epochs was investigated, while the $30{,}000$ training pairs were kept fixed. The final training error decreases continuously, from 0.00433 after five epochs to 0.00182 after fifty, but evaluation performance does not improve uniformly with it.

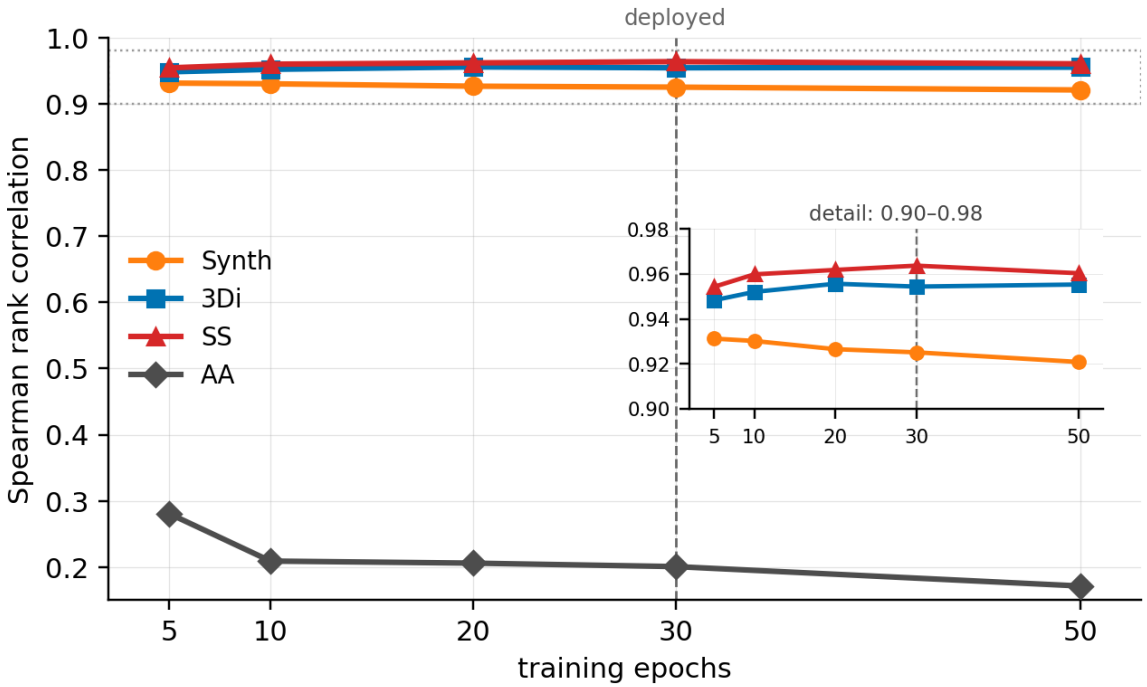


Figure 4.2: Overall Spearman correlation across epoch budgets. The dashed line marks the deployed thirty-epoch configuration. The inset magnifies the 0.90–0.98 band, within which Synth, 3Di and SS all lie.

No single epoch budget is therefore optimal across datasets or evaluation criteria: early checkpoints favour Synth retrieval and amino-acid ordering, whereas SS ordering and discrimination peak at the deployed thirty-epoch budget, and 3Di retrieval is almost insensitive to the budget.

High-range ordering is particularly stable. Between five and fifty epochs, high-range Spearman changes only from 0.868 to 0.863 on Synth, from 0.867 to 0.872 on 3Di and from 0.833 to 0.851 on SS. The high-similarity transfer reported in Chapter 3 is consequently not an artefact of the selected thirty-epoch budget. It persists across the complete epoch sweep and deteriorates clearly only when the pooling operation is removed, where the same three coefficients fall to 0.702, 0.829 and 0.700.

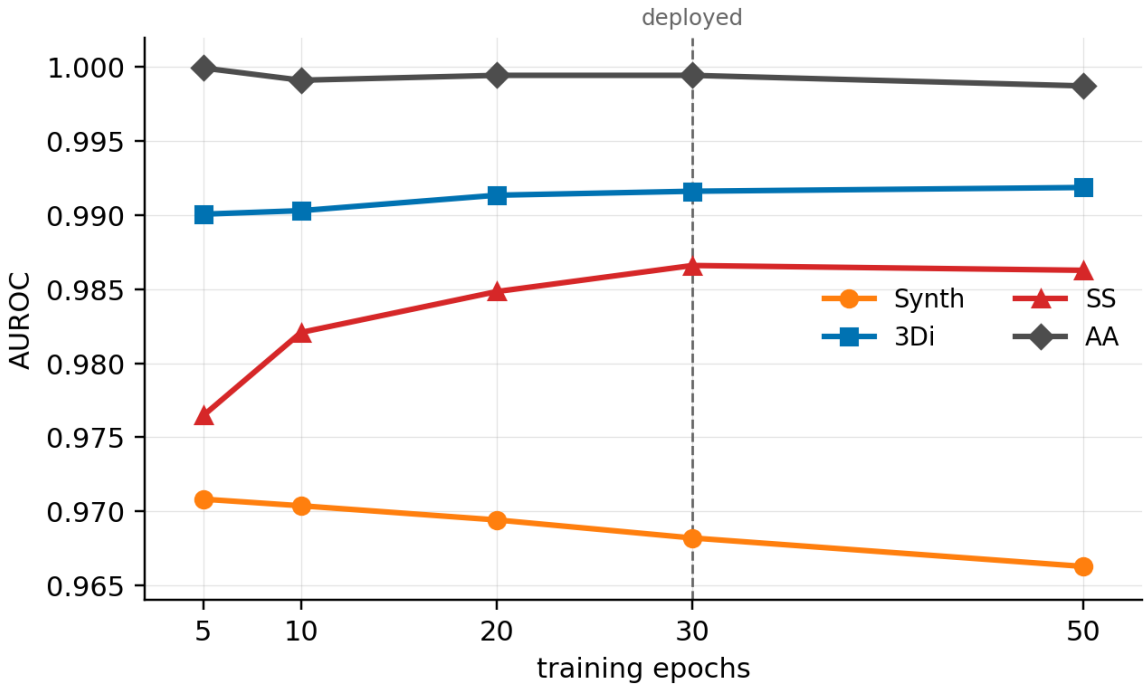


Figure 4.3: AUROC across epoch budgets. Values remain close to the upper limit across all configurations, so the vertical range is restricted. The AA estimates rest on five positive pairs among 1,211 negatives.

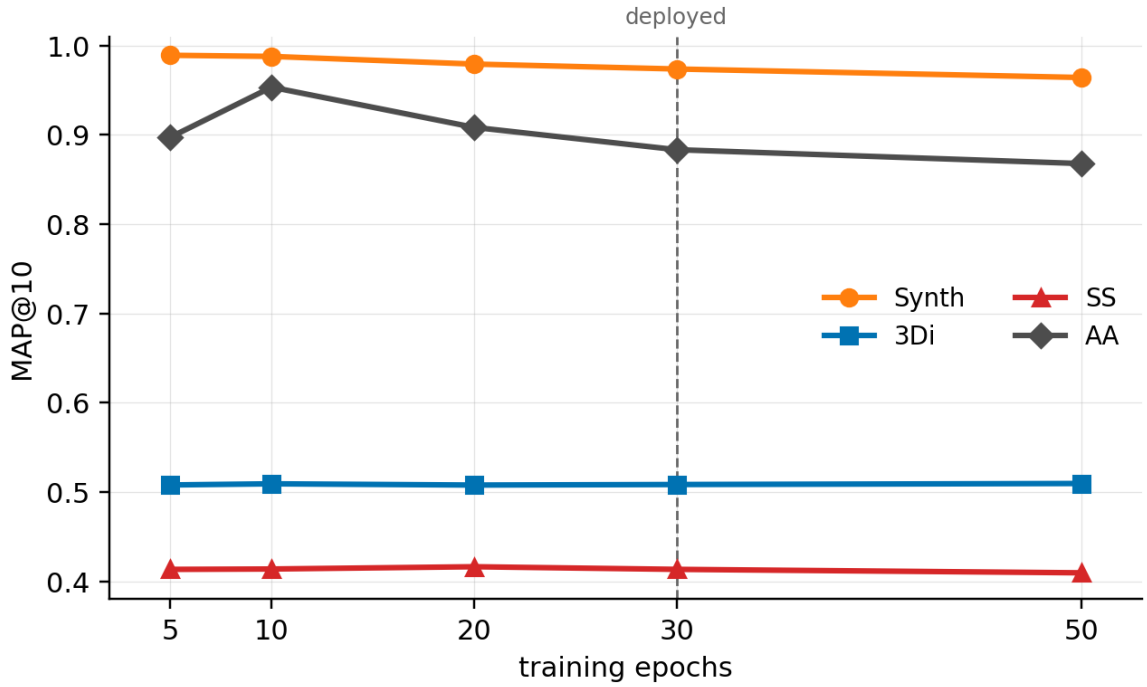


Figure 4.4: MAP@10 across epoch budgets. Synth retrieval declines with continued training, 3Di remains stable, and SS changes only slightly. AA rests on ten queries and consequently has limited support.

4.4.4 Effect of the Training-Set Size

At thirty epochs, as fixed in Chapter 2, the configuration is tested on 5,000, 10,000, 30,000 and 100,000 training pairs. Increasing the training set from 30,000 to 100,000 pairs raises the amino-acid Spearman correlation from 0.187 to 0.417. For thirty epochs each, the number

of optimiser steps scales with the training set, from $1{,}200$ at $5{,}000$ pairs to $23{,}460$ at $100{,}000$. This therefore measures the effect of the training-set size at a fixed epoch budget and does not separate it from the additional optimisation that comes with it, namely increasing the number of optimiser steps from $7{,}050$ to $23{,}460$.

Table 4.2: Spearman rank correlation and mean average precision at rank ten for four training-set sizes, all trained for thirty epochs. The optimiser step count is reported because it is not held constant across the rows. The AA retrieval values rest on ten queries with five relevant pairs among $1{,}211$ negatives.

		Spearman				MAP@10			
Training pairs	Steps	Synth	3Di	SS	AA	Synth	3Di	SS	AA
5,000	1,200	0.906	0.924	0.935	−0.067	0.942	0.489	0.393	0.750
10,000	2,370	0.905	0.936	0.931	0.029	0.936	0.491	0.400	0.632
30,000 (deployed)	7,050	0.925	0.955	0.964	0.187	0.975	0.506	0.411	0.835
100,000	23,460	0.950	0.965	0.974	0.417	0.999	0.537	0.423	1.000

The effect is smaller on the transferred representations. Overall Spearman rises by about 0.010 on both, from 0.955 to 0.965 on 3Di and from 0.964 to 0.974 on SS, against 0.230 on amino acids. Their retrieval results improve marginally: MAP@10 rises from 0.506 to 0.537 on 3Di and from 0.411 to 0.423 on SS. On Synth, overall Spearman increases from 0.925 to 0.950 and MAP@10 from 0.975 to 0.999. Additional training pairs therefore benefit amino-acid ordering and Synth retrieval most.

The configurations below $30{,}000$ pairs do not describe a useful learning curve. At $5{,}000$ pairs the amino-acid coefficient is negative, at -0.067, and at $10{,}000$ pairs it is 0.029, so the encoder carries almost no amino-acid ordering signal at either size. The defensible result is therefore the endpoint comparison between $30{,}000$ and $100{,}000$ pairs.

4.5 Limitations

The performance reported in this thesis should not be interpreted as an exact reproduction of the Levenshtein algorithm. Raw Levenshtein distance defines its own metric, whereas SNNEED represents each sequence as a point on a Euclidean unit sphere and compares two sequences using the Euclidean distance between their embeddings (Equation 2.4). Exact prediction would require the geometry of the relevant edit-distance space to be preserved by the Euclidean embedding.

The extent to which an embedding preserves a geometry is measured by its distortion. Ostrovsky and Rabani state the notion as follows: “Given two metric spaces (X_1, d_1) and (X_2, d_2), an embedding $\varphi : (X_1, d_1) \hookrightarrow (X_2, d_2)$ has distortion c if and only if distances are preserved up to a factor of c (and uniform scaling)” [39, Sec. 1].

$$\alpha\, d_1(x, y) \;\leq\; d_2\big(\varphi(x), \varphi(y)\big) \;\leq\; c\,\alpha\, d_1(x, y)$$

for all $x, y \in X_1$. A distortion of $c = 1$ corresponds to exact preservation of every distance up to that uniform scaling.

For SNNEED the source space is the set of strings with $d_1 = d_{\mathrm{Lev}}$, the Levenshtein distance. The target space is $\mathbb{R}^{128}$ with d_2 the Euclidean distance, and φ is the encoder, so that $\varphi(a) = e_a$ and $d_2(\varphi(a), \varphi(b)) = d_{\mathrm{Euc}}(a, b)$ of Equation 2.4.

Two quantities must be kept apart at this point. The predicted similarity $\widehat{s}(a, b)$ of Equation 2.5 is computed from the two embeddings, whereas the normalised Levenshtein similarity $s_{\mathrm{Lev}}(a, b)$ of Equation 2.1 is computed from the two strings themselves. If we assume no distortion and exact prediction for $\widehat{s}(a, b) = s_{\mathrm{Lev}}(a, b)$, then:

$$s_{\mathrm{Lev}}(a, b) \;=\; 1 - \frac{d_{\mathrm{Euc}}(a, b)}{2}, \qquad \text{hence} \qquad d_{\mathrm{Euc}}(a, b) \;=\; 2\big(1 - s_{\mathrm{Lev}}(a, b)\big),$$

and substituting Equation 2.1 for s_{Lev} leaves

$$d_{\mathrm{Euc}}(a, b) \;=\; \frac{2\, d_{\mathrm{Lev}}(a, b)}{\max\big(|a|, |b|\big)}.$$

Exact reproduction is the case $c = 1$, in which the two bounds of the definition coincide and $d_2 = \alpha\, d_1$. Comparing the two expressions, the scaling it would require is

$$\alpha \;=\; \frac{2}{\max\big(|a|, |b|\big)},$$

which is not a constant but changes with every pair. A uniform scaling of the kind the definition therefore cannot exactly reproduce the normalised target across sequences of differing length. Only if all sequences would share a same length L does the required scaling reduce to the constant $2/L$.

Known results on metric embeddings show that this cannot generally be achieved over the complete space of strings. Krauthgamer and Rabani proved that embedding the edit-distance metric on binary strings of length L into L_1 requires distortion $\Omega(\log L)$. They also note that L_2 embeds isometrically into L_1, so the same lower bound applies to embeddings into Euclidean space, independently of dimension [40]. Increasing the embedding dimension consequently cannot guarantee exact preservation of the complete edit-distance space, nor eliminate its worst-case distortion as sequence length increases.

These boundaries do not mean that useful approximation is impossible. Bourgain showed that any finite metric space containing N points can be embedded into Euclidean space with distortion proportional to $\log N$ [41]. Ostrovsky and Rabani subsequently constructed an edit-distance-specific embedding of binary strings into L_1 whose distortion grows subpolynomially with their length [39]. These results establish that approximate geometric representations of edit distance are possible, but they also emphasise that such representations generally introduce distortion instead of preserving every distance exactly.

A further limitation is introduced by the normalisation used in this thesis. A metric must satisfy symmetry and the triangle inequality, the last of which states that the direct path between two points is never longer than a path going through a third point. Dividing the Levenshtein distance by the length of the longer sequence does not preserve that property when sequences of different lengths are compared. The denominator depends on which pair is being compared, so routing through a longer intermediate string enlarges it on both legs and shrinks both normalised distances while the underlying edit costs still add. Writing

$n(a,b) = d_{\mathrm{Lev}}(a,b)/\max(|a|,|b|)$ for that normalised distance, the strings $a = \mathtt{ab}$, $b = \mathtt{aba}$ and $c = \mathtt{ba}$ give $n(a,c) = 2/2 = 1$, which exceeds $n(a,b) + n(b,c) = 1/3 + 1/3$. The normalised Levenshtein distance is therefore not a metric [42], and no isometric embedding of it exists into Euclidean space or into any other metric space. Exactness is out of reach here for a reason that precedes any distortion argument. This does not affect the fixed-length case used above, in which the normalisation is a constant rescaling of a genuine metric.

These limitations do not imply that every prediction error is caused by the incompatibility between edit-distance and Euclidean geometry. The experimental conclusions are also limited by the composition of the evaluation sets. Highly similar amino-acid pairs are rare under independent sampling, leaving only five positive pairs and ten eligible queries for the high-similarity AUROC, RMSE and MAP@10 evaluations. These results consequently have greater sampling uncertainty than the overall Spearman correlation, which rests on $1{,}216$ pairs. SNNEED's Spearman correlation, AUROC, MAP@10 and RMSE results should therefore be interpreted as measures of empirical approximation quality on the evaluated distributions. Overall, the predicted similarities should be interpreted as distribution-dependent approximations, not as an exact Euclidean reproduction of edit distance.

4.6 Future Work

SNNEED can be extended in several directions that could provide a more comprehensive overall assessment of its capabilities.

4.6.1 A Directly Comparable Baseline Comparison

ESM-2 was chosen as a baseline because general-purpose protein embeddings were shown to carry a sequence-similarity signal in their embedding geometry [15], although that evidence concerns the earlier ESM-1b model rather than ESM-2. In the present work that finding serves as motivation only. Evaluating SNNEED and ESM-2 under the sequence-similarity protocol of Fenoy et al., on the sequence sets they derive from CAFA3 [16], would allow the two to be compared directly rather than separately. One difference would have to be carried explicitly: that protocol takes BLASTp similarity [8] as its reference signal, whereas this thesis measures against normalised Levenshtein similarity throughout, so the two settings do not share a target.

The same comparison could be broadened. Of the twelve representations examined in that study, ESM ranked first and ProtTrans second, as reported in Section 1.4. Including a model from the ProtTrans family [13] would therefore add a second pretrained, task-agnostic representation built on an architecture and training objective distinct from ESM-2, and would place SNNEED's performance in the context of existing protein-retrieval approaches.

4.6.2 Improving Ranking and Retrieval Quality

The architecture decisions discussed in Section 4.4 indicate that the training-set size and the epoch budget should be considered together. A particularly relevant configuration would

use $100{,}000$ training pairs for ten epochs. At $100{,}000$ pairs, both Spearman correlation and retrieval improve, as reported in Table 4.2, while beyond ten epochs only SS improves clearly and 3Di marginally, as shown in Figures 4.2 and 4.3. This configuration would test whether broader coverage of the sequence space can improve generalisation and nearest-neighbour retrieval without requiring a longer training schedule.

4.6.3 Transfer Outside Bioinformatics

The generality of the approach should also be examined outside bioinformatics. The current evaluation is restricted to synthetic sequences and biological alphabets, whose symbol frequencies and transition probabilities differ from those of natural language. Applying SNNEED to a natural-language dataset would test whether the model can learn edit similarity for a qualitatively different sequence domain. Such an experiment could also investigate how tokenisation, alphabet size and non-uniform language structure affect the achievable approximation quality.

5 Conclusion

Can a relationship defined exactly by an algorithm become something a neural network can learn? The results of this thesis show that it can, although not without limits. Trained only on synthetic string pairs and their exact normalised Levenshtein similarities, SNNEED learned an embedding geometry that approximates the algorithm's input–output relationship. It did not reproduce the Levenshtein procedure or inherit its exact guarantees. Instead, it captured enough of the relationship to support ordering, high-similarity discrimination and, to a more limited extent, retrieval. Parts of that learned structure remained useful even when the encoder was applied to sequence representations it had never encountered during training.

The first research question asked whether a task-agnostic encoder already contains this global similarity signal and whether direct supervision can strengthen it. Frozen ESM-2 embeddings preserved a measurable relationship with normalised Levenshtein similarity despite ESM-2 having neither been trained for this target nor equipped with a task-specific prediction head. The signal was nevertheless compressed near the top of the cosine-similarity scale, where pairs with different Levenshtein similarities could receive nearly identical scores. SNNEED produced stronger overall ordering on every evaluated dataset. On 3Di, for example, its Spearman correlation reached 0.950 compared with 0.689 for ESM-2. Direct supervision therefore did not create the signal from nothing. It gave an incidental relationship finer, target-specific resolution.

The second research question asked how much of this learned relationship survives outside the synthetic training distribution. SNNEED preserved the broad structure particularly well. Its Spearman correlation among highly similar pairs remained close to 0.86 on Synth, 3Di and secondary structure, while its AUROC remained above 0.98 on both transferred representations. Its predictions also remained numerically close to the target scale. Retrieval revealed the boundary of this transfer. MAP@10 declined from 0.977 on Synth to 0.502 on 3Di and 0.405 on secondary structure. SNNEED therefore continued to distinguish highly similar pairs from less similar ones, but it preserved the detailed ordering of their neighbourhoods less reliably. The amino-acid collection cannot carry the same weight, since it contains only five high-similarity pairs and ten eligible queries, so its high-range results are descriptive rather than estimates.

The reference methods clarify why this result matters. Sequence length carried enough information to separate broad similarity regions, but almost none with which to retrieve the correct neighbours. The Dice coefficient over 3-grams performed almost perfectly on Synth, where the generator makes shared local patterns particularly distinctive, and it exceeded SNNEED on that dataset. That a fixed rule should win in its home conditions is not a shortfall of the encoder: where a local statistic already matches the data, learning is not required. Dice's performance fell sharply on 3Di and secondary structure as their symbol frequencies and transition patterns departed from those of the uniform synthetic strings. Learning helped exactly where fixed 3-gram overlap stopped suiting the data. Symbol frequencies and transitions therefore set the conditions under which Dice succeeds, and it is where those conditions fail that SNNEED's transfer shows.

A similar lesson emerged from the architecture experiments. Removing adaptive pooling produced a model with more than ten times as many parameters and a closer fit to the synthetic training pairs, yet worsened every reported evaluation metric on every dataset. How the encoder compressed sequence information mattered more than how closely it could fit the training pairs.

There is also a limit imposed by the target itself. The maximum-length normalisation used in this thesis does not satisfy the triangle inequality, so the target is not a metric at all and admits no isometric embedding into Euclidean space or into any other metric space, regardless of the embedding dimension. SNNEED can consequently offer only a distribution-dependent approximation of the algorithm's output relationship. The decline in retrieval quality is the clearest limitation of the present approximation. Preserving local neighbourhoods well enough for reliable search, and not only broad similarity relationships, remains open.

This thesis began with an apparent tension between algorithms and neural networks. The Levenshtein algorithm provides an exact, reproducible procedure whose validity does not depend on the distribution from which its inputs were drawn. SNNEED offers no equivalent guarantee and did not demonstrate that it recovered the exact algorithmic procedure. It did however learn an approximation of the relationship the algorithm defines, one that remained useful when the input representation changed. Algorithms and neural networks therefore do not need to be treated as alternatives. The algorithm defines the relationship precisely and supplies the exact target, while the neural network learns how that relationship can be expressed in a representation that can be reused on unseen inputs. The tension is resolved not by replacing the algorithm, but by using its precision to guide what the network learns.

Acknowledgements

I would like to sincerely thank **Prof. Michael Schroeder** and **Ferras El-Hendi** for their continuous guidance, support, feedback and ideas over the last months. Inviting me into their group allowed me to immerse myself in the field of neural networks and bioinformatics. I also would like to thank **Prof. Michael Färber** for taking the time to review and support my work.

Lastly, I would like to thank everyone in the Schröder Group for welcoming me in and providing help and interesting discussions. I am grateful for the opportunity to work with such a great team and to learn from them.

Bibliography

[1] Petar Veličković and Charles Blundell. “Neural Algorithmic Reasoning”. In: *Patterns* 2.7 (2021), p. 100273. DOI: 10.1016/j.patter.2021.100273.

[2] Robert Geirhos et al. “Shortcut learning in deep neural networks”. In: *Nature Machine Intelligence* 2 (2020), pp. 665–673. DOI: 10.1038/s42256-020-00257-z.

[3] Louis-Pascal A. C. Xhonneux et al. “How to transfer algorithmic reasoning knowledge to learn new algorithms?” In: *Advances in Neural Information Processing Systems 34 (NeurIPS 2021)*. 2021. URL: https://proceedings.neurips.cc/paper/2021/hash/a2802cade04644083dcde1c8c483ed9a-Abstract.html.

[4] Bonnie Berger, Michael S. Waterman, and Yun William Yu. “Levenshtein Distance, Sequence Comparison and Biological Database Search”. In: *IEEE Transactions on Information Theory* 67.6 (2021), pp. 3287–3294. DOI: 10.1109/TIT.2020.2996543.

[5] V. I. Levenshtein. “Binary codes capable of correcting deletions, insertions, and reversals”. In: *Soviet Physics Doklady* 10.8 (1966), pp. 707–710.

[6] Robert A. Wagner and Michael J. Fischer. “The String-to-String Correction Problem”. In: *Journal of the ACM* 21.1 (1974), pp. 168–173. DOI: 10.1145/321796.321811.

[7] Michel van Kempen et al. “Fast and accurate protein structure search with Foldseek”. In: *Nature Biotechnology* 42 (2024), pp. 243–246. DOI: 10.1038/s41587-023-01773-0.

[8] Stephen F. Altschul et al. “Basic local alignment search tool”. In: *Journal of Molecular Biology* 215.3 (1990), pp. 403–410. DOI: 10.1016/S0022-2836(05)80360-2.

[9] Gonzalo Navarro. “A guided tour to approximate string matching”. In: *ACM Computing Surveys* 33.1 (2001), pp. 31–88. DOI: 10.1145/375360.375365.

[10] Martin Steinegger and Johannes Söding. “MMseqs2 enables sensitive protein sequence searching for the analysis of massive data sets”. In: *Nature Biotechnology* 35.11 (2017), pp. 1026–1028. DOI: 10.1038/nbt.3988.

[11] Jacob Devlin et al. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *Proceedings of NAACL-HLT 2019*. 2019, pp. 4171–4186.

[12] Alec Radford et al. “Learning Transferable Visual Models From Natural Language Supervision”. In: *Proceedings of the 38th International Conference on Machine Learning (ICML)*. Vol. 139. Proceedings of Machine Learning Research. 2021, pp. 8748–8763. URL: https://proceedings.mlr.press/v139/radford21a.html.

[13] Ahmed Elnaggar et al. “ProtTrans: Toward Understanding the Language of Life Through Self-Supervised Learning”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44.10 (2022), pp. 7112–7127. DOI: 10.1109/TPAMI.2021.3095381.

[14] Zeming Lin et al. “Evolutionary-scale prediction of atomic-level protein structure with a language model”. In: *Science* 379.6637 (2023), pp. 1123–1130. DOI: 10.1126/science.ade2574.

[15] Emilio Fenoy, Alejandro A. Edera, and Georgina Stegmayer. “Transfer learning in proteins: evaluating novel protein learned representations for bioinformatics tasks”. In: *Briefings in Bioinformatics* 23.4 (2022), bbac232. DOI: 10.1093/bib/bbac232.

[16] Naihui Zhou, Yuxiang Jiang, Timothy R. Bergquist, et al. “The CAFA challenge reports improved protein function prediction and new functional annotations for hundreds of genes through experimental screens”. In: *Genome Biology* 20 (2019), p. 244. DOI: 10.1186/s13059-019-1835-8.

[17] Alexander Rives et al. “Biological structure and function emerge from scaling unsupervised learning to 250 million protein sequences”. In: *Proceedings of the National Academy of Sciences* 118.15 (2021), e2016239118. DOI: 10.1073/pnas.2016239118.

[18] Xinyan Dai et al. “Convolutional Embedding for Edit Distance”. In: *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2020, pp. 599–608. DOI: 10.1145/3397271.3401045.

[19] Mubarak G. Abdu-Aguye et al. “Adaptive Pooling Is All You Need: An Empirical Study on Hyperparameter-insensitive Human Action Recognition Using Wearable Sensors”. In: *2020 International Joint Conference on Neural Networks (IJCNN)*. 2020. DOI: 10.1109/IJCNN48605.2020.9207082.

[20] Gabriele Corso et al. “Neural Distance Embeddings for Biological Sequences”. In: *Advances in Neural Information Processing Systems 34 (NeurIPS 2021)*. 2021. URL: https://proceedings.neurips.cc/paper/2021/hash/9a1de01f893e0d2551ecbb7ce4dc963e-Abstract.html.

[21] Jane Bromley et al. “Signature Verification using a ‘Siamese’ Time Delay Neural Network”. In: *Advances in Neural Information Processing Systems 6 (NeurIPS 1993)*. 1993. URL: https://proceedings.neurips.cc/paper/1993/hash/288cc0ff022877bd3df94bc9360b9c5d-Abstract.html.

[22] Masahiro Ohtomo, Atsuhiro Takasu, and Tatsuya Akutsu. “Computing Hamming Distance and Levenshtein Distance Using ReLU Neural Networks”. In: *IEEE Access* 13 (2025), pp. 210089–210101. DOI: 10.1109/ACCESS.2025.3639297.

[23] Max Bachmann. *RapidFuzz: Rapid fuzzy string matching in Python*. Version 3.14.5. Python library. URL: https://github.com/rapidfuzz/RapidFuzz.

[24] Diederik P. Kingma and Jimmy Ba. “Adam: A Method for Stochastic Optimization”. In: *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*. arXiv:1412.6980. 2015.

[25] Ian Sillitoe et al. “CATH: increased structural coverage of functional space”. In: *Nucleic Acids Research* 49.D1 (2021), pp. D266–D273. DOI: 10.1093/nar/gkaa1079.

[26] Lee R. Dice. “Measures of the Amount of Ecologic Association Between Species”. In: *Ecology* 26.3 (1945), pp. 297–302. DOI: 10.2307/1932409.

[27] Adam Paszke et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library”. In: *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*. 2019. URL: https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html.

[28] Charles R. Harris et al. “Array programming with NumPy”. In: *Nature* 585.7825 (2020), pp. 357–362. DOI: 10.1038/s41586-020-2649-2.

[29] Pauli Virtanen et al. “SciPy 1.0: fundamental algorithms for scientific computing in Python”. In: *Nature Methods* 17.3 (2020), pp. 261–272. DOI: 10.1038/s41592-019-0686-2.

[30] Fabian Pedregosa et al. “Scikit-learn: Machine Learning in Python”. In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830. URL: https://www.jmlr.org/papers/v12/pedregosa11a.html.

[31] Thomas Wolf et al. “Transformers: State-of-the-Art Natural Language Processing”. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, 2020, pp. 38–45. DOI: 10.18653/v1/2020.emnlp-demos.6.

[32] John D. Hunter. “Matplotlib: A 2D Graphics Environment”. In: *Computing in Science & Engineering* 9.3 (2007), pp. 90–95. DOI: 10.1109/MCSE.2007.55.

[33] Wes McKinney. “Data Structures for Statistical Computing in Python”. In: *Proceedings of the 9th Python in Science Conference*. 2010, pp. 56–61. DOI: 10.25080/Majora-92bf1922-00a.

[34] Jesse Davis and Mark Goadrich. “The Relationship Between Precision-Recall and ROC Curves”. In: *Proceedings of the 23rd International Conference on Machine Learning (ICML)*. 2006, pp. 233–240. DOI: 10.1145/1143844.1143874.

[35] Václav Chvátal and David Sankoff. “Longest Common Subsequences of Two Random Sequences”. In: *Journal of Applied Probability* 12.2 (1975), pp. 306–315. DOI: 10.2307/3212444.

[36] Marcos Kiwi, Martin Loebl, and Jiří Matoušek. “Expected Length of the Longest Common Subsequence for Large Alphabets”. In: *Advances in Mathematics* 197.2 (2005), pp. 480–498. DOI: 10.1016/j.aim.2004.10.012.

[37] Satya N. Majumdar and Sergei Nechaev. “Exact Asymptotic Results for the Bernoulli Matching Model of Sequence Alignment”. In: *Physical Review E* 72.2 (2005), p. 020901. DOI: 10.1103/PhysRevE.72.020901.

[38] Craig A. Tracy and Harold Widom. “Level-Spacing Distributions and the Airy Kernel”. In: *Communications in Mathematical Physics* 159.1 (1994), pp. 151–174. DOI: 10.1007/BF02100489.

[39] Rafail Ostrovsky and Yuval Rabani. “Low Distortion Embeddings for Edit Distance”. In: *Journal of the ACM* 54.5 (2007), p. 23. DOI: 10.1145/1284320.1284322.

[40] Robert Krauthgamer and Yuval Rabani. “Improved Lower Bounds for Embeddings into L_1”. In: *SIAM Journal on Computing* 38.6 (2009), pp. 2487–2498. DOI: 10.1137/060660126.

[41] Jean Bourgain. “On Lipschitz Embedding of Finite Metric Spaces in Hilbert Space”. In: *Israel Journal of Mathematics* 52.1–2 (1985), pp. 46–52. DOI: 10.1007/BF02776078.

[42] Yujian Li and Bo Liu. "A Normalized Levenshtein Distance Metric". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 29.6 (2007), pp. 1091–1095. DOI: 10.1109/TPAMI.2007.1078.

A Data Distributions

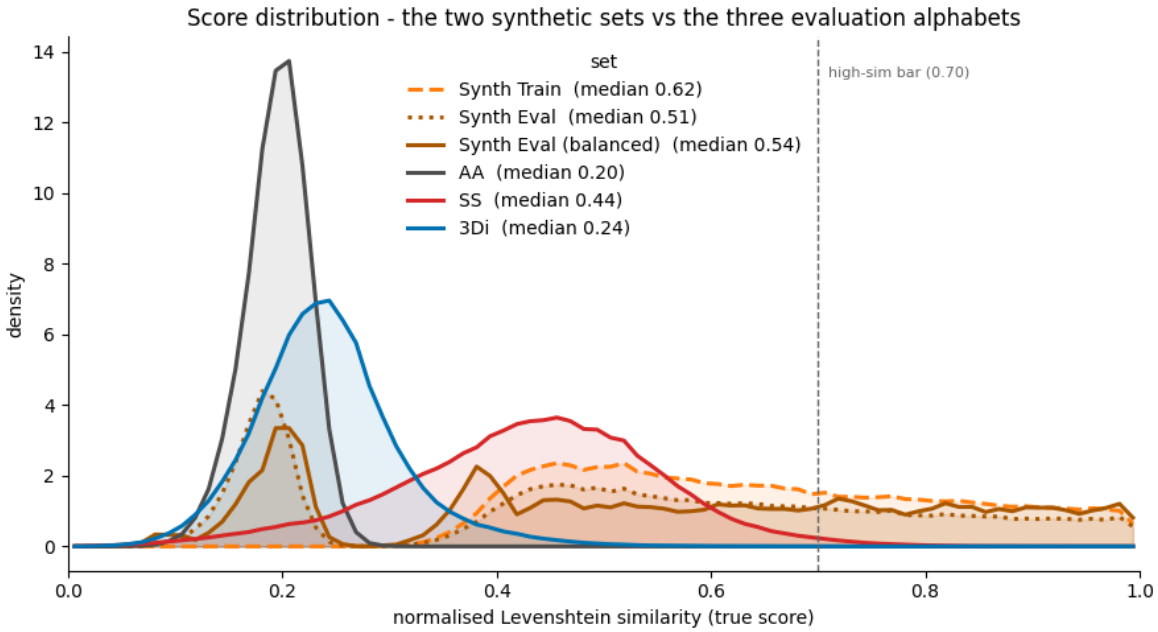


Figure A.1: Distribution of normalised Levenshtein similarity for the synthetic training set, the synthetic evaluation set before and after decile balancing, and the three protein-derived evaluation datasets. The curves for the protein-derived datasets are computed over all pairs of their respective search collections; the synthetic curves over their generated pairs. The dashed line marks the boundary of the high range at 0.70.

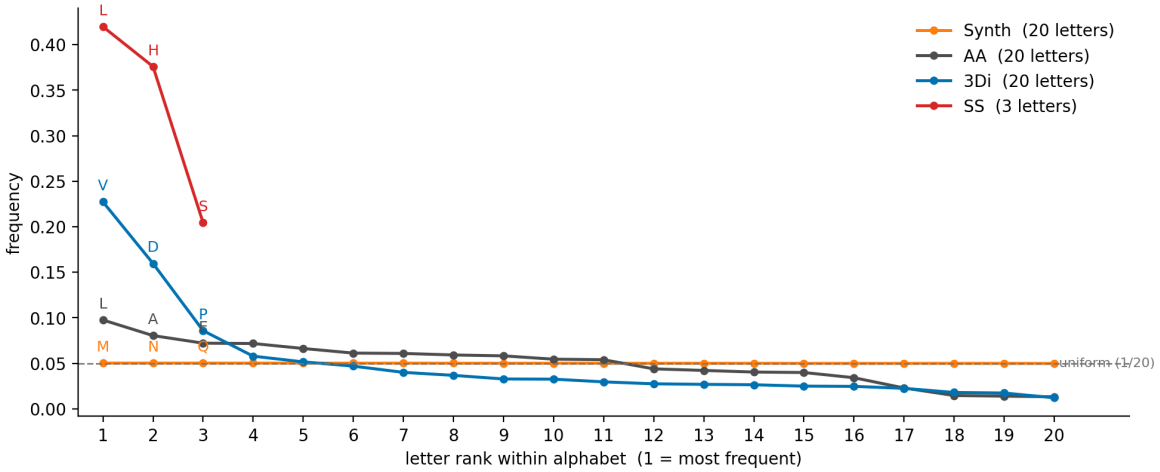


Figure A.2: Letter-frequency profiles of the four evaluation datasets. For each dataset the symbols of its own alphabet are sorted from the most to the least frequent, so that the profiles remain comparable although the alphabets differ in size. The dashed line marks the uniform frequency $1/20$.

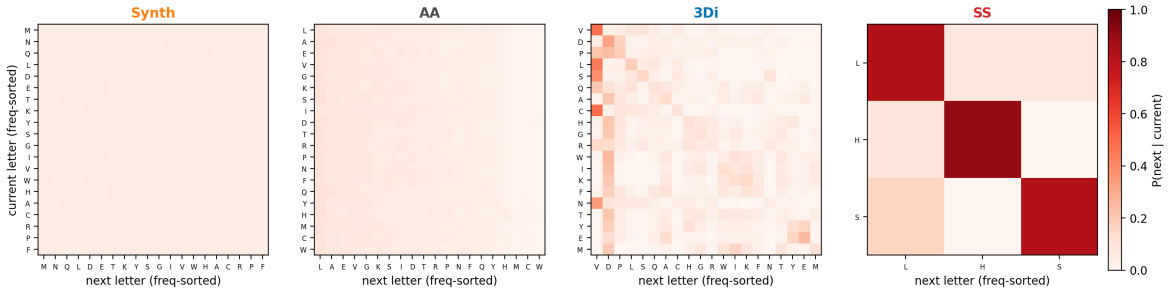


Figure A.3: First-order transition probabilities $P(\text{next} \mid \text{current})$ for the four evaluation datasets. Rows and columns are sorted by symbol frequency, and each row is a probability distribution conditioned on the current symbol.

B Composition of the Evaluation Sets

The decile-balanced evaluation sets of Section 2.6.2 are dependent on two factors: the number of candidate pairs sampled per collection, and the maximum number of pairs retained per decile interval. This appendix reports what each of them does. The synthetic evaluation dataset is governed by a further quantity, the number of pairs generated for it, and is treated separately in Section B.3.

B.1 Per-Interval Supply and the Bound

Table B.1 gives the supply of pairs available per decile interval for all four datasets. The size of the retained set is on the far right. For the three protein-derived collections it is obtained the following way: from 200,000 sampled pairs, with all available high-similarity pairs added on top, up to four hundred pairs from each decile are taken. The synthetic dataset is constructed pairwise and has no collection to sample from, so its candidates are the 28,000 generated pairs themselves (Section 2.6.2); only the bound is applied. Either way at most four thousand pairs are retained, an effort made to distribute all four datasets as evenly over the normalised Levenshtein similarity scale as their individual collections allow. The four datasets differ sharply nonetheless.

The supply of the amino-acid collection stops after the fourth interval and resumes only with the five high-similarity pairs of Table 2.6, which fall in the eighth and ninth intervals. The resulting set is therefore the smallest of the four. Sampling over a bigger pool does not change which intervals it fills. Table B.3 reports how differently sized pools influence how many pairs can be retained per interval. At one million candidate pairs the amino-acid set gains about fifty pairs and fills the same intervals. Below roughly 185,000 candidates, however, the lowest interval can no longer supply the bound and the set loses an interval. The secondary-structure collection supplies more than four hundred pairs in every interval, giving the most balanced set of the four.

The bound itself is a compromise rather than an optimum. Table B.2 reports the effect of different bounds. Lowering or raising the bound comes with different trade-offs: lowering it makes the retained set more evenly distributed but decreases the sample size, and raising it does the reverse. The direction cannot however be chosen freely. Above four hundred

Table B.1: Candidate pairs available in each similarity interval, before the per-interval bound is applied. Interval k covers $[k/10, (k+1)/10)$. The counts include the high-similarity pairs added to the candidates. The final column gives the size of the evaluation set obtained after the bound.

Dataset	0	1	2	3	4	5	6	7	8	9	Decile-balanced
Synth	48	6,414	1,540	945	4,437	4,033	3,277	2,696	2,372	2,238	3,648
3Di	1,868	48,859	117,523	26,520	4,496	624	151	4,168	1,440	348	3,699
SS	1,020	6,721	19,298	46,530	69,037	44,622	22,379	558,751	52,818	1,885	4,000
AA	405	109,425	90,144	11	0	0	0	3	2	0	1,216

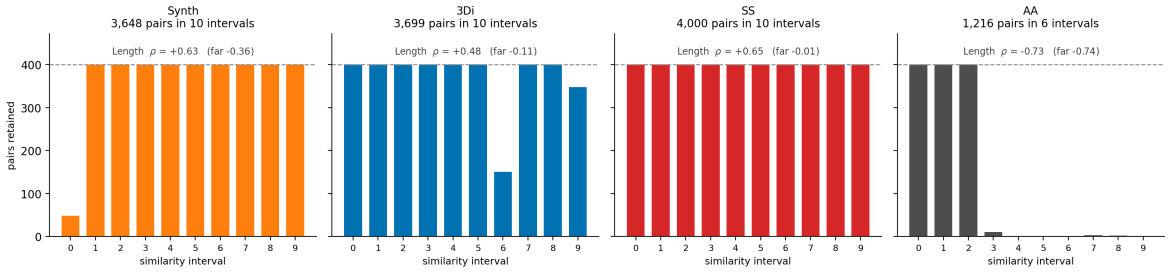


Figure B.1: Pairs retained in each similarity interval, for each evaluation dataset, after the per-interval bound of Section 2.6.2 is applied. Each panel is annotated with that dataset's whole-set and far-range correlation for the length baseline.

and five the amino-acid collection retains two intervals at the bound instead of three, four hundred and five being the supply of its lowest interval. The same quantity fixes the number of candidate pairs from the other side. The amino-acid collection supplies a little over four hundred pairs in its lowest interval at 200,000 candidates, which is at once why the bound cannot be raised much further and why the number of candidates cannot be lowered much further. The two are not independent choices.

What remains uneven is not something a bound can repair. It is fixed by the scarcest interval of each collection, which holds two pairs for amino acids, one hundred and fifty-one for 3Di and forty-eight for the synthetic dataset. The secondary-structure collection supplies at least four hundred pairs everywhere and is therefore the only one of the four that the bound balances completely. A bound equalises the intervals that could exceed it and leaves the rest as they are, so the residual differences describe the collections rather than the protocol.

Table B.2: Effect of the per-interval bound on the four evaluation datasets. Each entry gives the size of the resulting evaluation set, with the number of intervals retained at the bound in brackets. The value used throughout is four hundred.

Bound	Synth	3Di	SS	AA
50	498 (9)	500 (10)	500 (10)	166 (3)
100	948 (9)	1,000 (10)	1,000 (10)	316 (3)
200	1,848 (9)	1,951 (9)	2,000 (10)	616 (3)
400	3,648 (9)	3,699 (8)	4,000 (10)	1,216 (3)
800	7,248 (9)	6,723 (7)	8,000 (10)	2,021 (2)
1,600	13,733 (7)	12,163 (6)	15,420 (9)	3,621 (2)
3,200	22,639 (4)	20,431 (5)	28,505 (8)	6,821 (2)
10,000	28,000 (0)	43,095 (3)	79,626 (7)	20,421 (2)

B.2 The Number of Candidate Pairs

Above the point at which the amino-acid collection fills its lowest interval, the number of candidates has little further effect. Raising it fivefold leaves the secondary-structure evaluation set unchanged at four thousand pairs in ten equally filled intervals, since no interval of that collection falls short at any of the values examined, and adds about fifty pairs to the amino-acid set without changing the intervals it fills. Only the 3Di collection changes in kind, and only at the largest value examined, where its seventh interval reaches the bound.

Table B.3: Effect of the number of candidate pairs on the three protein-derived evaluation sets. Each entry gives the size of the resulting evaluation set, with the number of intervals retained at the bound in brackets. The candidates are drawn independently at each size, so the row for 200,000 differs from Table B.1 by a few pairs. The synthetic dataset is constructed pairwise and is not sampled from a collection.

Candidate pairs	3Di	SS	AA
100,000	3,538 (7)	4,000 (10)	1,030 (2)
200,000	3,701 (8)	4,000 (10)	1,219 (3)
400,000	3,778 (8)	4,000 (10)	1,233 (3)
700,000	3,886 (8)	4,000 (10)	1,246 (3)
1,000,000	3,951 (9)	4,000 (10)	1,269 (3)

B.3 Size of the Synthetic Evaluation Dataset

The synthetic evaluation dataset is not sampled from a collection, so neither the number of candidates nor the per-interval bound is what limits it. What limits it is how many pairs are generated, and in which of the two ways.

The synthetic evaluation dataset is composed of dependent sequence pairs (Section 2.4.2), which populate the scale from roughly 0.3 upward. To account for the low similarity range, independent pairs are generated: altered copies reach that range only rarely, since a pair of strings over the same alphabet is unlikely to fall below the similarity two independently drawn strings already reach by chance. Table B.4 shows what that costs. Without independent pairs the two lowest intervals are empty and the third holds two pairs, so the evaluation set has no low-similarity end at all, which poses a problem especially for a Spearman correlation ranking.

Beyond that the exact number matters little. From four thousand independent pairs onward nine of the ten intervals are retained at the bound, and raising the count from four thousand to twenty thousand changes the size of the evaluation set by about two percent. The lowest interval is never filled at any of the values examined, reaching one hundred and seven pairs at twenty thousand against a bound of four hundred. That limit is a property of the alphabet rather than of the number of pairs generated, and is examined in Section 4.3.

Table B.4: Effect of the number of independently generated pairs on the three lowest intervals, on the number of intervals retained at the bound and on the size of the synthetic evaluation set.

Independent pairs	Interval 0	Interval 1	Interval 2	At bound	Evaluation set
0	0	0	2	7	2,802
2,000	10	1,604	388	8	3,598
4,000	22	3,204	776	9	3,622
8,000	48	6,414	1,540	9	3,648
20,000	107	15,848	4,047	9	3,707

C The Length Constraint

Every sequence used in this work is restricted to a length between 50 and 200 symbols. The bound was introduced in the first experiment that used CATH data and remained unchanged in every experiment after it. Its original justification was the expectation that pairs of very unequal length would mislead the encoder during training, since sequences of extremely unequal length are dissimilar by construction. The normalised similarity of a pair cannot exceed the ratio of the shorter length to the longer one. Applying the interval retains roughly $70\,\%$ of the domains in the source collection. The padding width followed from the interval, and was set to match the upper bound that had already been chosen. Padding to a common width is itself required, because the encoder requires fixed-size inputs.

Table C.1 reports what each bound removes. Of the $14{,}907$ domains available before filtering, $1{,}119$ fall below 50 symbols and a further $3{,}289$ exceed 200, leaving $10{,}499$. The longest domain in the collection is $1{,}202$ symbols. These reductions change the supply of pairs, but they do not by themselves change the evaluation sets, because the per-interval bound of Section 2.6.2 caps the contribution of any similarity interval at four hundred pairs and the reductions fall almost entirely above that cap. The construction of these evaluation sets is described in Appendix B. The amino-acid collection is the scarcest, and its scarcity is not a length problem, since removing the constraint entirely adds but a single further high-similarity amino-acid pair.

Table C.1: Sequences removed by each length bound, of the $14{,}907$ domains available before filtering. The same domain set underlies all three representations.

	Sequences	% of collection
below 50 symbols	1,119	7.5
retained, [50, 200]	10,499	70.4
above 200 symbols	3,289	22.1

The constraint was nevertheless tested directly, by rebuilding the search collections with both bounds removed. The number of eligible 3Di queries rises from 347 to 817, a factor of 2.35, and the number of secondary-structure queries from $10{,}002$ to $13{,}572$. These counts are properties of the collection and do not depend on the encoder. In none of the configurations examined did the additional queries translate into better retrieval quality than the restricted collection provides.

The reason for retaining the constraint lies elsewhere. Within the interval the ratio between the longer and the shorter sequence of a pair cannot exceed $4{:}1$, so no pair can obtain a normalised similarity above 0.25 on length grounds alone; in the unrestricted collection the ratio reaches roughly $130{:}1$, and the corresponding limit falls to 0.008. A wide ratio range therefore makes length itself a strong predictor of the target. This effect can be measured: On the unrestricted amino-acid collection the length-ratio baseline reaches a Spearman rank correlation of 0.652 and exceeds every trained configuration, and $1{,}200$ of the $1{,}216$ amino-acid evaluation pairs lie in the far range, where that predictor is most easily exploited. Retrieval within the restricted collection shows the converse. There the length-ratio baseline obtains MAP@10 values of 0.010, 0.008 and 0.016 on Synth, 3Di and secondary structure (Section 3.2), so within the deployed interval length alone does not identify a relevant neighbour, although it still orders pairs coarsely across similarity ranges (Section 3.1). The constraint's function is therefore to hold the length-ratio range narrow enough to limit the extent to which length can be used as a shortcut, so that the embedding is shaped by the normalised Levenshtein target rather than by the ratio of two lengths. The constraint is retained for that reason, and may be relaxed for 3Di and secondary structure by a study concerned with those two representations alone.

D The Null Datasets

A similarity score can only be read as high or low against the score that two unrelated strings of the same kind would obtain. That reference point is measured rather than assumed. Two null datasets are constructed for each of the four evaluation datasets, each of $20{,}000$ pairs, and both take their lengths from the collection they refer to. What they show is discussed in Section 4.3.

The *independent* null generates two new strings for every pair. The two lengths are drawn from the length distribution of the collection, and every symbol is then drawn uniformly and independently from that dataset's alphabet. No sequence of the collection enters other than through its length, so the resulting distribution is determined by the alphabet and the lengths alone. In both nulls the sequence pairs are scored with the same exact normalised Levenshtein similarity as every other pair in this work, and each floor is reported as the median over its $20{,}000$ pairs. The curve corresponding to this null is the one drawn beneath the observed distributions in Figures D.1 and D.2.

The *shuffled* null instead permutes the symbols of the real sequences. It preserves each sequence's length and its symbol composition and destroys only the order in which the symbols appear, which is what makes the difference between the two nulls a measurement of composition. For the synthetic dataset the two agree to three decimal places, its symbols being generated with uniform probabilities already, so that permuting them changes nothing the independent null had not assumed.

The two nulls are not plotted together. The synthetic, amino-acid and 3Di collections share one alphabet and therefore one independent floor, which is why a single curve serves all three; under the shuffled null their floors separate slightly and would require three.

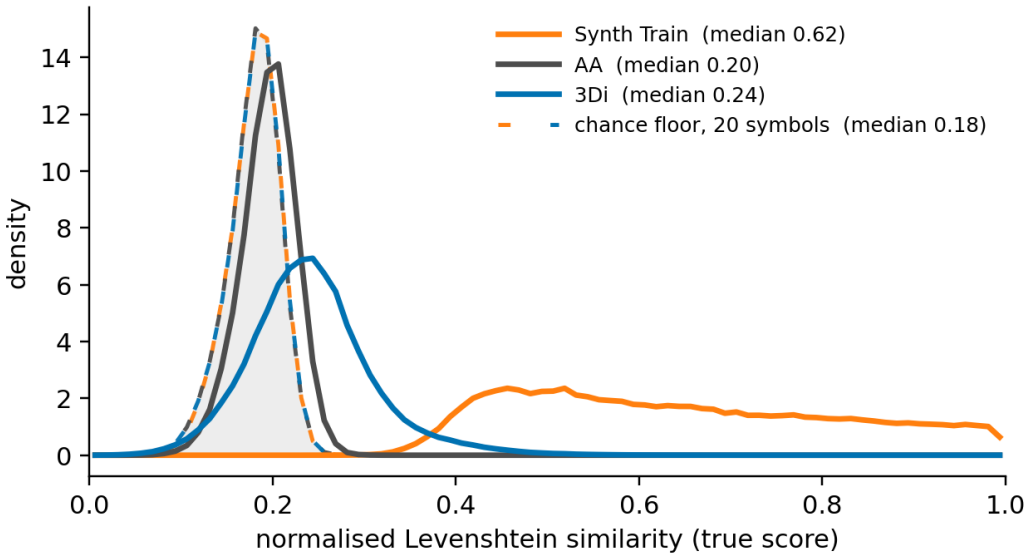


Figure D.1: Distribution of exact normalised Levenshtein similarity for the three datasets over the twenty-symbol alphabet, each shown against the independent null of that alphabet. The amino-acid and 3Di curves are computed over all pairs of their collections, with medians of 0.20 and 0.24; the synthetic curve has a median of 0.62. The three floors coincide and are drawn as one dashed curve in the three colours, at a median of 0.183. The synthetic curve is spread across the range by construction, the number of edits applied to each seed string being drawn uniformly. Its rise towards the low end of that range is follows however the distribution pull described here.

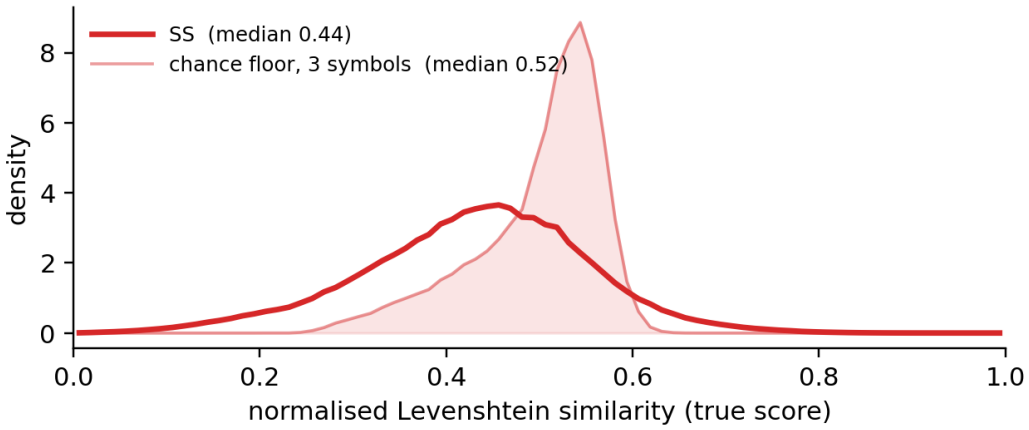


Figure D.2: Distribution of exact normalised Levenshtein similarity for secondary structure, over all pairs of its collection, against the independent null of the three-symbol alphabet. The horizontal range is that of Figure D.1. The observed median of 0.44 lies below the floor's median of 0.520.

Statement of authorship

I hereby declare that I have written this thesis, entitled *Embedded Edit Distance: Levenshtein Approximation with a Siamese Neural Network for String Similarity Search*, independently and without undue assistance from third parties. No sources or aids other than those named in this document have been used, and every passage taken from another work, whether quoted literally or in substance, has been identified as such.

AI-based tools, including large language models, were used in support of this work. Their use extended to exploring the literature and the problem space, structuring and refining arguments, assistance with programming and debugging, discussion of results, and linguistic revision of the manuscript. Beyond this, the schematic diagrams shown in Figures 2.1, 2.2, 2.3 and 2.4 were generated with ChatGPT; they illustrate the method and contain no measured data.

The design of the experiments, the computational runs, the processing of the data, the evaluations and the scientific conclusions drawn from them are my own. Every external or AI-generated suggestion, idea, result, figure or other content that entered this thesis has been critically reviewed by me, either tested or verified, and appropriately referenced.

I further declare that this thesis has not been submitted, in whole or in part, to any other examination authority for the award of an academic degree, and that its printed and electronic versions are identical. I am aware that violations of this declaration may lead to subsequent withdrawal of the academic degree.

Dresden, 7th September 2026

Melissa Analytis